



DIPLOMATERVEZÉSI FELADAT

Pásztori Péter Benjámín
Mérnök-informatikus hallgató részére

Egészséges életmódot támogató webalkalmazás fejlesztése ASP.NET MVC és Angular alapokon

Az egészséges életmód fenntartása és a táplálkozással kapcsolatos adatok nyomon követése napjainkban egyre nagyobb kihívást jelent. A tudatos étkezés és a rendszeres testmozgás összehangolása, valamint az ehhez szükséges információk rögzítése és visszakeresése gyakran bonyolult vagy időkieszt jelent. A modern webfejlesztési környezetek – mint például az ASP.NET MVC és az Angular – olyan lehetőségeket kínálnak, amelyekkel korszerű, felhasználóbarát, alkalmazások hozhatók létre. Az így elkészült szoftver segítséget nyújthat a mindennapi étkezések, receptek, tápanyagok és mozgásformák rugalmas vezetésében, és megfelelő elemzési lehetőségeket kínálhat az életmód célok eléréséhez (például fogyás vagy tömegnövelés). A hallgató feladata egy olyan webalkalmazás megtervezése és megvalósítása, amely a regisztrált felhasználók életmódjának nyomon követését és támogatását szolgálja.

A hallgató feladatának a következőkre kell kiterjednie:

- Felhasználókezelés és profilbeállítás
- Alapanyag- és receptkezelés:
 - Alapanyag rögzítése tápanyag-információkkal (fehérje, zsír, szénhidrát, kalória).
 - Receptek létrehozása és szerkesztése, amelyek alapanyagokból épülnek fel.
- Az elfogyasztott ételek rögzítése a naplóba, és a kalória-/tápanyagtartalom összesítése.
- Sporttevékenységek, mozgásformák felvétele.
- Célkövetés és visszajelzés: Az aktuális napi tápanyagbevitel (kalória, makrotápanyagok) folyamatos összevetése a profilban beállított céllal, vizuális visszajelzéssel.
- Előzmények és statisztikák:
 - Korábbi napok adatai (étkezések, mozgás).
 - A testsúly alakulásának grafikus megjelenítése.

Tanszéki konzulens: Dr. Kővári Bence, egyetemi docens

Budapest, 2025. február 26.

Dr. Charaf Hassan
egyetemi tanár
tanszékvezető





Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Pásztori Péter Benjámin

EGÉSZSÉGES ÉLETMÓDOT TÁMOGATÓ WEBALKALMAZÁS FEJLESZTÉSE ASP.NET MVC ÉS ANGULAR ALAPOKON

KONZULENS

Dr. Kővári Bence András

BUDAPEST, 2025

Tartalomjegyzék

Összefoglaló	7
Abstract.....	8
1 Bevezetés	9
1.1 A digitális egészségtudatosság kora	9
1.2 A probléma felvetése és a fejlesztésem célja	10
1.3 A megoldás szakmai motivációja	10
2 Irodalomkutatás, technológiák, hasonló alkotások bemutatása.....	12
2.1 Technikai háttér	12
2.2 Szerver oldali technológiák.....	12
2.2.1 ASP.NET Core.....	12
2.2.2 Hitelesítés / Autorizáció.....	13
2.2.3 E-mail szolgáltatás	14
2.2.4 API dokumentáció és kliensgenerálás (Swagger és NSwag)	14
2.2.5 Entity Framework	15
2.3 Kliens oldali technológiák	15
2.3.1 Angular	16
2.3.2 TypeScript.....	16
2.3.3 HTML és CSS.....	16
2.3.4 Bootstrap.....	17
2.4 Adattárolás	17
2.4.1 Microsoft SQL Server.....	17
2.5 Fejlesztőkörnyezet	18
2.5.1 Visual Studio Code	18
2.5.2 Visual Studio 2022.....	18
2.5.3 Munkafolyamat követése	19
2.5.4 Verziókezelés (Git és GitHub).....	19
3 A feladatkiírás pontosítása és részletes elemzése	20
3.1 Funkcionális követelmények	20
3.2 Nem-funkcionális követelmények	21
3.3 Szerepkörök és használati esetek	21
3.4 Rendszerarchitektúra	23

3.5	Adatbázis-tervezés	24
3.6	Fejlesztési környezet és eszközök.....	25
4	Önálló munka bemutatása	26
4.1	Rendszertervezés és Adatmodell	26
4.1.1	Adatbázis koncepció	26
4.1.2	Az Adatmodell részletezése:.....	27
4.1.3	DTO-k szerepe	29
4.2	A Backend megvalósítása (ASP.NET Core)	29
4.2.1	Architektúra és Tervezési minták	30
4.2.2	Hitelesítés és biztonság	30
4.2.3	Felhasználói profil és Célok kezelése	32
4.2.4	Ételek	33
4.2.5	Sporttevékenységek kezelése.....	34
4.2.6	Receptkezelés:	34
4.2.7	Üzleti logika - A Naplózás magja.....	36
4.2.8	Adatszolgáltatás a vizualizációhoz (Grafikon és Naptár).....	37
4.2.9	API Dokumentáció:	37
4.3	A Frontend megvalósítása.....	38
4.3.1	Kliens oldali architektúra és Modularitás	38
4.3.2	Navigáció és Útvonal-védelem (Routing & Security).....	41
4.3.3	Kommunikáció a Backenddel (API Layer)	43
4.3.4	Felhasználókezelés (Account Module).....	46
4.3.5	Adatkezelés és Törzsadatok (CRUD)	48
4.3.6	Komplex űrlapkezelés: Receptkészítő	51
4.3.7	A Naplózás megvalósítása (Daily Note).....	53
5	Önálló munka értékelése, mérések, tesztelés, eredmények bemutatása	64
5.1	Automatizált tesztelés és kódminőség	64
5.2	A kész rendszer bemutatása (Felhasználói foratókönyvek).....	65
5.2.1	Regisztráció és Profilalkotás.....	65
5.2.2	Egy teljes nap rögzítése (Naplózás).....	67
5.2.3	Receptkészítés és felhasználás.....	67
5.2.4	Hosszú távú követés (Grafikonok)	68
5.3	Nem-funkcionális követelmények teljesülése	68
5.4	Eredmények és a követelmények teljesülése	68

6	Összefoglaló.....	70
7	Irodalomjegyzék.....	71

HALLGATÓI NYILATKOZAT

Alulírott **Pásztori Péter Benjám**n, szigorló hallgató kijelentem, hogy ezt a diplomatervet meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző, cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy hitelesített felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2025. 12. 01.

.....
Pásztori Péter Benjám

Összefoglaló

A 21. századi mozgásszegény életmód és a feldolgozott élelmiszerek globális egészségügyi kihívásai miatt felértékelődött a digitális egészségtudatosság. A felhasználók egyre inkább igénylik a táplálkozást és aktivitást integráltan kezelő szoftvereket, mivel a jelenlegi piaci megoldások gyakran nem nyújtanak kellő rugalmasságot az egyéni célok és receptek kezelésében.

Diplomamunkám célja egy korszerű, integrált webalkalmazás megvalósítása volt az egészséges életmód támogatására. A fejlesztés során „Full Stack” megközelítést alkalmaztam: a rendszert a nagyvállalati környezetben elterjedt ASP.NET Core és a skálázható Angular keretrendszerek ötvöztetésével építettem fel.

A szerveroldalon Microsoft SQL Server alapú, Code-First rendszert alakítottam ki, kiemelt figyelmet fordítva az adatok integritására. Implementáltam a „Recalculation Chain” üzleti logikát, amely valós időben, matematikailag helyesen frissíti a napi statisztikákat, a biztonságot pedig ASP.NET Core Identity, JWT hitelesítés és Mailjet e-mail szolgáltatás garantálja.

A kliensoldali hatékonyságot NSwag kódgenerátorral növeltem, a felhasználói élményt pedig reaktív űrlapokkal és Chart.js alapú vizualizációval tettem teljessé. A rendszer megbízhatóságát és az üzleti logika helyességét xUnit és InMemory Database alapú automatizált integrációs tesztekkel igazoltam.

Az elkészült alkalmazás sikeresen teljesíti a funkcionális követelményeket, stabil felületet biztosítva a felhasználóknak. A projekt során szerzett tapasztalataim megerősítették, hogy a tiszta architektúra és az automatizált tesztelés a minőségi szoftverfejlesztés alapkövei. A jövőben a rendszert natív mobilalkalmazással és vonalkód-leolvasóval tervezem tovább fejleszteni.

Abstract

Ide jön a ½-1 oldalas angol nyelvű összefoglaló, amelynek szövege a Diplomaterv Portálra külön is feltöltésre kerül. Ez a magyar nyelvű összefoglaló angolra fordított változata.

1 Bevezetés

1.1 A digitális egészségtudatosság kora

A 21. század társadalmát jelentős életmódbeli változások jellemzik, amelyek új kihívások elé állítják az egyéneket az egészségmegőrzés területén. Az iparosodott világban általánossá vált az ülőmunka, amely a fizikai aktivitás drasztikus csökkenésével jár. Ezzel párhuzamosan a mindennapi élet felgyorsult tempója és a folyamatos stressz, valamint a könnyen elérhető, magasan feldolgozott élelmiszerek elterjedése hozzájárult a krónikus megbetegedések, mint például az elhízás, a 2-es típusú diabétesz és a szív- és érrendszeri problémák népbetegséggé válásához.

Ebben a megváltozott környezetben az egészségtudatosság és a prevenció szerepe felértékelődött. Az emberek egyre inkább felismerik azt, hogy az egészségük megőrzéséért aktívan kell tenniük, és nem csupán a már kialakult betegségek kezelésére támaszkodhatnak.

Ez a proaktív szemlélet hívta életre azt a technológiai robbanást, amely az öngondoskodást és az egészségügyi prevenciót digitális alapokra helyezte. A szélessávú internet elterjedése, majd az okostelefonok és a viselhető okoseszközök (mint az okosórák és fitneszkarkötők) mindennapivá válása gyökeresen megváltoztatta a viszonyunkat a saját testünk működéséhez.

Megjelent az úgynevezett "Quantified Self" (Számszerűsített én) mozgalom, amelynek központi gondolata, hogy az egyének képesek adatokat gyűjteni saját életmódjukról – legyen szó lépésszámról, alvási ciklusokról, pulzusról vagy éppen tápanyagbevitelről. Ezek az eszközök lehetővé tették, hogy a korábban rejtett vagy csak laboratóriumi körülmények között mérhető biometrikus adatok bárki számára azonnal elérhetővé és értelmezhetővé váljanak. A technológia demokratizálta az adathozzáférést, és ezzel párhuzamosan megnőtt az igény az olyan alkalmazások iránt, amelyek ezt a nyers adatözönt képesek kontextusba helyezni és cselekvési tervvé alakítani. A szoftveres megoldások már nem csupán passzív adatgyűjtők, hanem aktív partnerekké váltak az életmódváltás folyamatában. Lehetővé teszik a felhasználók számára, hogy személyre szabott célokat tűzzenek ki – legyen szó napi kalóriabevitelről, makro tápanyag-egyensúlyról, a testsúly kívánt alakulásáról vagy a heti edzésterhelésről.

Ezek a platformok vizuális visszajelzésekkel, statisztikákkal és napi összesítőkkal támogatják a felhasználói motivációt, és kézzelfoghatóvá teszik a haladást. Kiemelt fontosságúvá vált az egészséges életmód két legalapvetőbb pillére: a táplálkozás és a fizikai aktivitás nyomon követése. Míg a viselhető eszközök az aktivitást egyre nagyobb pontossággal, automatikusan mérik, addig a táplálkozás naplózása – annak komplexitása miatt – továbbra is jelentős felhasználói beavatkozást és precíz szoftveres támogatást igényel.

1.2 A probléma felvetése és a fejlesztésem célja

Az informatikai megoldások jelentős támaszt nyújthatnak az egészségtudatosabb élet kialakításában, hiszen a digitális naplózás, a valós idejű visszajelzések és a statisztikai elemzések segítségével a felhasználók tudatosabb döntéseket hozhatnak. A diplomatervemben - modern technológiák (ASP.NET Core és Angular) felhasználásával - egy olyan webalkalmazás fejlesztését tűztem ki célul, amely alkalmas az egészséges életmód támogatására.

A dolgozatomban az alkalmazás fejlesztési folyamatát és architektúráját mutatom be, külön figyelmet fordítottam a rendszer felépítésére, a technológiai döntésekre, valamint a felhasználói funkciók megvalósítására. A rendszerem képes a felhasználó napi étkezéseinek, aktivitásainak, testsúlyának és célkitűzéseinek kezelésére, valamint ezek összegzésére és vizualizálására.

1.3 A megoldás szakmai motivációja

Az egészségtudatosság terjedésével sok felhasználó párhuzamos eszközökben, táblázatokban és különálló mobilalkalmazásokban próbálja követni az étkezéseit és aktivitását, ami időigényes és pontatlanságot szül. A napi makrók kézi összesítése, a receptek és egyedi ételek vegyes kezelése, valamint a visszajelzések hiánya gyakran megtöri a következetes naplózást.

Szükség van egy olyan, egyszerűen használható, mégis bővíthető webes megoldásra, amely egy helyen kezeli az étkezések, receptek és aktivitások rögzítését, valós idejű tápanyag-összegzéssel támogatja a döntéshozatalt, és vizualizációval teszi követhetővé a haladást. A témám szakmailag is indokolt, mert modern, rétegezett webes

architektúrára épít, és gyakorlati példán mutatja be, hogyan illeszthetők össze a kliensoldali kényelem, a típushű API-kommunikáció és az adatvédelem követelményei.

1.4. A dolgozat felépítése

A bevezetést követően, a 2. fejezetben a fejlesztéshez felhasznált technológiákat és eszközöket mutatom be. Részletezem a szerveroldali (ASP.NET Core, Entity Framework Core) és a kliensoldali (Angular, TypeScript) keretrendszereket, valamint indoklom a választásukat. Ismertetem továbbá az adattároláshoz választott Microsoft SQL Server és a verziókövetéshez használt Git rendszer szerepét a projektben.

A 3. fejezetben a tervezési folyamatot és a követelmények elemzését dokumentálom. Definiálom a rendszerrel szemben támasztott funkcionális és nem-funkcionális elvárásokat, valamint a felhasználói szerepköröket. Ezt követően bemutatom az általam tervezett rendszerarchitektúrát és az adatbázis koncepcionális tervét, amelyekre az implementációt alapoztam.

A dolgozatom központi részét a 4. fejezet alkotja, ahol az önálló fejlesztői munkámat tárgyalom. Itt ismertetem a végleges adatmodellt, majd részletesen bemutatom a backend üzleti logikájának és a frontend felhasználói felületének megvalósítását. Kitérek a legfontosabb algoritmusokra – például a tápanyag-számításra és a receptkezelésre –, valamint a kliens és szerver közötti biztonságos kommunikáció kiépítésére.

Az 5. fejezetben az elkészült alkalmazást értékelem. Bemutatom a tesztelési folyamatot és annak eredményeit, valamint elemzem, hogy a megvalósított rendszer mennyiben felel meg a feladatkiírásban és a tervezés során rögzített követelményeknek.

Végezetül a 6. fejezetben összegzem a munkám eredményeit és a projekt során szerzett szakmai tapasztalataimat. Ugyanitt vázolom fel az alkalmazás lehetséges továbbfejlesztési irányait, javaslatot téve a jövőbeni funkcióbővítésekre.

2 Irodalomkutatás, technológiák, hasonló alkotások bemutatása

2.1 Technikai háttér

A webalkalmazásomat két fő komponensre bontottam: a backendre és a frontendre. A szerveroldali logikát (backend) ASP.NET Core technológiával készítettem el, mivel ezt egy modern, moduláris és nyílt forráskódú keretrendszernek tartom. A kliensoldali megjelenítéshez (frontend) az Angular 19 keretrendszert választottam, amely erős JavaScript alapjaival különösen előnyösnek bizonyult az egyoldalas (SPA) alkalmazásom fejlesztéséhez.

A backend oldalon Microsoft SQL Server adatbázist alkalmaztam, amellyel az Entity Framework Core ORM révén kommunikálok. A rendszerben RESTful API-kon keresztül biztosítottam a kapcsolatot a frontend felé, így szigorúan szétválasztottam a kliens és a szerveroldali logikát. Data Transfer Object (DTO) használatával garantáltam, hogy a backend és a frontend közötti adattovábbítás tömör és biztonságos legyen.

A frontend komponensben az NSwag eszköz segítségével generált TypeScript service osztályokat használtam, amelyek automatikusan illeszkednek az általam írt backend API-khoz és DTO-khoz. A felhasználói felületen Angular Material és Bootstrap komponenseket alkalmaztam a responszív és esztétikus megjelenés érdekében. A felhasználókezelést az ASP.NET Identity segítségével oldottam meg, amelyet kiegészítettem jelszó-visszaállítási, email-megerősítési és token-alapú hitelesítési lehetőségekkel. Az email küldést a Mailjet API segítségével valósítottam meg.

2.2 Szerver oldali technológiák

2.2.1 ASP.NET Core

A diplomatervem szerveroldali logikáját, a RESTful API-t, ASP.NET Core keretrendszerrel valósítottam meg. Tekintettel arra, hogy a feladatkiírásom is ezt a technológiát jelölte meg, egyértelmű volt a választás. Ez a modern, platformfüggetlen

keretrendszer tökéletes alapot adott egy robusztus, nagy teljesítményű webes API elkészítéséhez.

A fejlesztés során kihasználtam a keretrendszer számos beépített funkcióját, amelyek közül a projektem szempontjából a legfontosabbak a következők voltak:

- Kestrel: A beépített, nagy teljesítményű HTTP-kiszolgáló.
- Integrált függőséginjektálás (Dependency Injection): Ezt alkalmaztam a szolgáltatások (pl. RecipeService, DailyNoteService) és a DbContext kontroller-szintű kezelésére, ami a kódot tisztán és tesztelhetően tartotta.
- Minimális API-k és MVC: Bár a keretrendszer támogatja a minimális API-kat, én a hagyományos Model-View-Controller (MVC) felépítést választottam (kontroller osztályokkal), mert a projekt komplexitása (pl. a DailyNoteController-ben lévő több végpont) így átláthatóbb maradt.
- Integráció az Angularral: A beépített mechanizmusok (pl. CORS beállítások) megkönnyítették a kliensoldali alkalmazással való zökkenőmentes kommunikációt.
- Beépített biztonsági funkciók: Ezekre építettem a hitelesítési és autorizációs logikát. [1]

2.2.2 Hitelesítés / Autorizáció

A felhasználók biztonságos kezelését és az API végpontok védelmét ASP.NET Identity segítségével oldottam meg.

A kliensoldali Angular alkalmazás és a backend közötti állapotmentes (stateless) kommunikációhoz JSON Web Token (JWT) alapú hitelesítést implementáltam. A Microsoft.AspNetCore.Authentication.JwtBearer csomagot használtam a kliens által küldött tokenek validálására.

A felhasználók, szerepkörök (Roles) és a jelszavak (hash-elt formában) tárolását a Microsoft.AspNetCore.Identity.EntityFrameworkCore csomag tette lehetővé, amely automatikusan integrálta az Identity modellt az Entity Framework adatbázis-sémámba. A tokenek tényleges generálását a JWTService-ben, a IdentityModel.Tokens.JWT csomag segítségével valósítottam meg. [2] [3]

2.2.3 E-mail szolgáltatás

A felhasználói regisztráció megerősítéséhez és az "elfelejtett jelszó" funkcióhoz szükség volt egy megbízható e-mail küldő szolgáltatásra. Erre a célra a Mailjet platformját választottam.

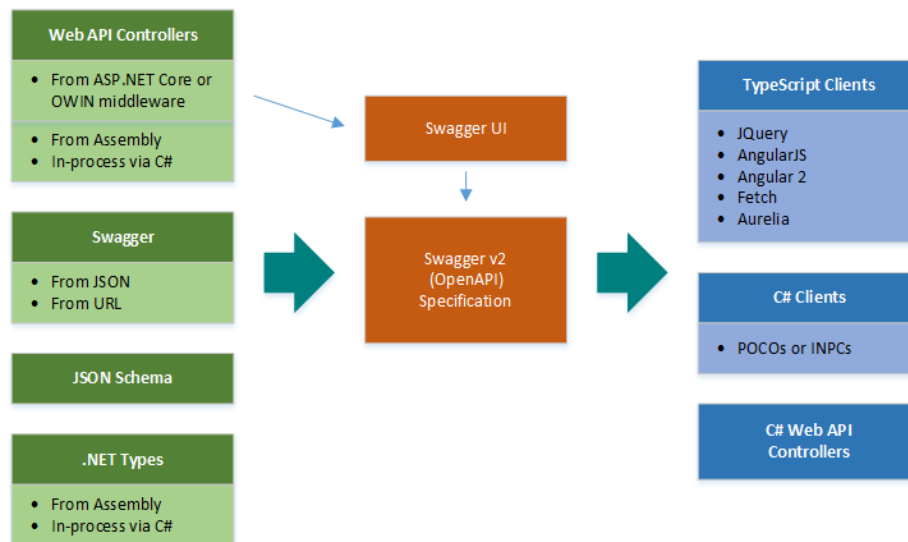
A HealthyAPI/Services/EmailService.cs osztályomban implementáltam azt a logikát, amely a Mailjet Representational State Transfer (REST) alapú API-ján keresztül küldi ki a leveleket. Az API hívások törzse JavaScript Object Notation (JSON) formátumú adatokat tartalmaz. [4]

2.2.4 API dokumentáció és kliensgenerálás (Swagger és NSwag)

A frontend és a backend fejlesztésének szinkronizálása és a típusbiztos kommunikáció érdekében kulcsfontosságú volt az API pontos dokumentálása. Erre az NSwag eszköztárat használtam.

Az NSwag a C# kontroller osztályaim (HealthyAPI/Controllers) és DTO-im (HealthyAPI/DTOs) alapján futásidőben automatikusan generálta az OpenAPI (Swagger) specifikációt. Ez a swagger.json fájl szolgált alapul a böngészőben futó interaktív Swagger UI felülethez, ami nagy segítség volt a fejlesztés és a tesztelés során.

A munkafolyamat másik kulcslépéseként az NSwag (az ad.nswag konfigurációs fájl alapján) képes volt ebből az OpenAPI specifikációból legenerálni a teljes TypeScript kliens osztályt (Client/src/app/shared/models/Nswag generated/NswagGenerated.ts) az Angular alkalmazásom számára. Így a frontend oldalon típusbiztosan, fordítási időben ellenőrizhető metódusokkal hívhattam meg a backend végpontokat, elkerülve a manuális http hívásokból adódó hibákat. [5]



1. ábra Nswag generálás folyamata

2.2.5 Entity Framework

Az alkalmazás adatainak perzisztens tárolásához Microsoft SQL Server adatbázist választottam. Az adatbázis és a C# objektumorientált modellek (HealthyAPI/Models) közötti leképezését az Entity Framework Core (EF Core) keretrendszerrel oldottam meg.

Ez a modern, objektum-relációs leképező (ORM) lehetővé tette, hogy tiszta C# osztályokkal definiáljam az adatbázis sémáját, és ne kelljen manuálisan SQL parancsokat írnom. A lekérdezéseket Language-Integrated Query (LINQ) segítségével fogalmaztam meg a szolgáltatási rétegben (pl. a RecipeService-ben). Az adatbázis-séma változásait és verziókövetését az EF Core Migrations funkciójával kezeltem. [3]

2.3 Kliens oldali technológiák

A felhasználói felületet (frontend) modern, egyoldalas alkalmazásként (SPA - Single Page Application) terveztem meg. Ennek lényege, hogy a böngésző inicializáláskor csak egyszer tölti be a keretrendszert és az alkalmazás vázát, a továbbiakban pedig a felhasználói interakciók során dinamikus cserélem a tartalmat, az oldal teljes újra töltése nélkül. A fejlesztés során komponensalapú architektúrát követtem, ami lehetővé tette a kód újra felhasználhatóságát és a könnyebb karbantarthatóságot.

2.3.1 Angular

A kliens oldali logikát az Angular 19 verziójára építve valósítottam meg. A projektet modulokra és komponensekre bontottam a Client/src/app könyvtárban.

- Routing modul (AppRoutingModule): Ezzel oldottam meg az oldalak közötti navigációt. A src/app/app-routing.module.ts fájlban definiáltam az útvonalakat (pl. /daily-note, /recipes), amelyekhez a megfelelő komponenseket rendeltem hozzá, így a felhasználó számára az oldalváltás azonnali és zökkenőmentes.
- Függőséginjektálás (Dependency Injection): Ezt a tervezési mintát használtam a szolgáltatások (Services) beemelésére. Például az AccountService-t a konstruktoron keresztül injektáltam a komponensekbe, hogy központosítva kezeljem a bejelentkezési logikát és a felhasználói állapotot.
- Reaktív űrlapok (Reactive Forms): A szigorúbb validációt igénylő felületeket, mint például a regisztrációt (RegisterComponent), ezzel a módszerrel kezeltem. Ez lehetővé tette a beviteli mezők komplex validációját (pl. kötelező mezők, e-mail formátum, jelszóerősség) még azelőtt, hogy az adatokat elküldtem volna a szervernek. [6]

2.3.2 TypeScript

A TypeScript a JavaScript típusos superhalmaza, amely statikus típusellenőrzést tesz lehetővé fejlesztési időben, ezáltal csökkentve a futás idejű hibák számát.

Bár a böngészők JavaScriptet futtatnak, a forráskódot TypeScript nyelven írtam. Ez lehetővé tette számomra a szigorú típusosság érvényesítését az egész alkalmazásban. Különösen nagy hasznát vettem az NSwag eszközzel generált DTO-k esetében. A src/app/shared/models/Nswag generated/NswagGenerated.ts fájlban található interfészek (pl. IDailyNoteResponseDto, IRecipeResponseDto) biztosították, hogy a frontend adatmodellje mindig pontosan szinkronban legyen a backend API válaszaival. Így, ha a backend megváltozott, a fordító azonnal jelezte a hibát a kliens kódban. [7]

2.3.3 HTML és CSS

A HTML (HyperText Markup Language) a weboldalak szerkezetét leíró nyelv, míg a CSS (Cascading Style Sheets) a megjelenésért és formázásért felelős stílusleíró

nyelv. Az alkalmazás vázát HTML5 sablonok segítségével definiáltam. Minden Angular komponenshez (pl. `daily-note.component.html`) külön HTML fájlt rendeltem, így tisztán szétválasztottam a megjelenítést (View) az üzleti logikától. A stílusozáshoz CSS3-at használtam. Kihasználtam az Angular komponens-szintű stílusizolációját (ViewEncapsulation), ami azt jelenti, hogy az egyes komponensekhez írt CSS szabályok (pl. a `daily-note.component.css`-ben) csak arra az adott komponensre érvényesek, nem "szivárognak át" az alkalmazás többi részére, így elkerültem a stílusütközéseket. [8]

2.3.4 Bootstrap

A felhasználói felület alapjait a Bootstrap 5 keretrendszerrel integráltam a projektbe, mivel ez a világ legnépszerűbb nyílt forráskódú eszközkészlete a reszponzív, mobil-első weboldalak fejlesztéséhez. Annak érdekében, hogy az alkalmazás minden eszközön (mobiltelefonon és asztali gépen is) megfelelően jelenjen meg, a rácsrendszer (Grid system) osztályait (pl. `container`, `row`, `col-md-6`) alkalmaztam a HTML sablonokban az elrendezések kialakítására. Ez modern színvilágot és egységes tipográfiát kölcsönzött a felületnek. A felhasználói visszajelzésekhez (pl. sikeres mentés) nem külső toast-könyvtárat, hanem az `ngx-bootstrap` modális ablakait (`BsModalService`) alkalmaztam.

2.4 Adattárolás

2.4.1 Microsoft SQL Server

A Microsoft SQL Server egy robusztus, vállalati szintű relációs adatbázis-kezelő rendszer (RDBMS). Az alkalmazás strukturált adatainak (felhasználók, naplóbejegyzések, receptek) perzisztens tárolására Microsoft SQL Server adatbázist (a fejlesztés során LocalDB vagy SQLExpress példányt) alkalmaztam. A választást az indokolta, hogy ez az adatbázis-kezelő natívan és rendkívül hatékonyan integrálható a .NET környezetbe és az általam használt Entity Framework Core ORM-be. A kapcsolatot a `HealthyAPI/appsettings.json` fájlban konfiguráltam a `DefaultConnection` kapcsolati karakterlánccal.

2.5 Fejlesztőkörnyezet

A fejlesztés során a hatékonyság maximalizálása érdekében két különböző, az adott technológiához legjobban illeszkedő fejlesztőkörnyezetet (IDE) használtam párhuzamosan.

2.5.1 Visual Studio Code

A kliens oldali (Angular) fejlesztéshez a Visual Studio Code kódszerkesztőt választottam. Ez a szerkesztő kiváló támogatást nyújt a TypeScript nyelvhez és a webes technológiákhoz. A fejlesztés során nagyban támaszkodtam a következő funkciókra:

- Beépített terminál: Ezen keresztül futtattam az Angular CLI parancsokat (pl. `ng generate component`, `ng serve`) közvetlenül a szerkesztőből.
- Bővítmények (Extensions): Az Angular Language Service bővítmény használata segített a HTML sablonokban lévő hibák azonnali, gépelés közbeni felismerésében és az automatikus kiegészítésben.
- Fájlkezelés: A Client mappa hierarchiájának átlátható kezelése és a `package.json` függőségek menedzselése itt történt.

2.5.2 Visual Studio 2022

A szerver oldali (ASP.NET Core) logikát a nagyvállalati környezetben szabványnak számító Visual Studio 2022 integrált fejlesztőkörnyezetben valósítottam meg. A választást az indokolta, hogy ez az eszköz nyújtja a legteljesebb támogatást a C# nyelvhez és a .NET ökoszisztémához.

- Solution kezelés: A `HealthyAPI.sln` solution fájl összefogta a projekt összes elemét, a kontrollerektől az adatbázis-migrációkig.
- NuGet csomagkezelő: Ezen keresztül telepítettem és frissítettem a backend működéséhez szükséges könyvtárakat (pl. `Microsoft.EntityFrameworkCore`, `NSwag.AspNetCore`).
- Hibakeresés (Debugging): A beépített hibakeresővel lépésről lépésre tudtam követni a backend kód futását, a változók értékét és a kivételeket, ami elengedhetetlen volt a komplexebb üzleti logika (pl. `DailyNoteService` számításai) ellenőrzéséhez. [9]

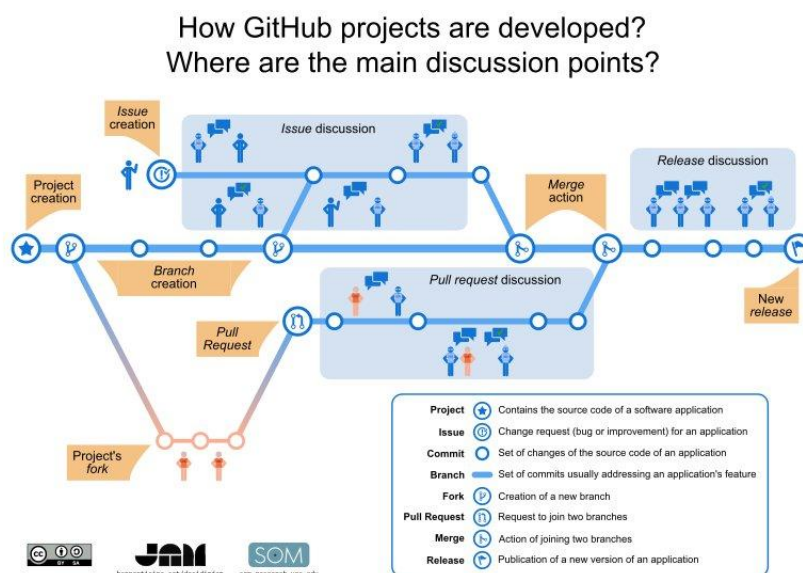
2.5.3 Munkafolyamat követése

2.5.4 Verziókezelés (Git és GitHub)

A forráskód biztonságos tárolását, verziókövetését és a változtatások dokumentálását a Git verziókezelő rendszerrel oldottam meg. A távoli tároló (remote repository) hosztolására a GitHub szolgáltatást vettem igénybe.

A fejlesztés során az alábbi munkafolyamatot követtem:

- Commit-ok: A logikailag összetartozó módosításokat (pl. "Új recept létrehozása funkció hozzáadása") rendszeresen rögzítettem (commit), így a fejlesztés bármely pontjára visszatérhettem volna hiba esetén.
- Mappastruktúra védelme: A gyökerkönyvtárban és a Client mappában elhelyezett .gitignore fájlok segítségével biztosítottam, hogy a felesleges, generált fájlok (pl. node_modules, bin, obj mappák, felhasználói beállítások) ne kerüljenek fel a felhőbe, csak a tiszta forráskód.
- Szinkronizáció: A lokális gépen végzett munkámat rendszeresen szinkronizáltam (push) a GitHub tárolóval, ami nemcsak biztonsági mentésként szolgált, hanem lehetővé tette a kód hordozhatóságát is a különböző fejlesztői gépek között



2. ábra GitHub munkafolyamat

3 A feladatkiírás pontosítása és részletes elemzése

3.1 Funkcionális követelmények

A funkcionális követelmények határozzák meg a rendszerem elvárt viselkedését és szolgáltatásait. A feladatkiírás és a saját terveim alapján az alábbi funkciókat definiáltam:

- Felhasználókezelés:
 - Regisztráció e-mail cím és jelszó megadásával.
 - Biztonságos bejelentkezés (JWT token alapú hitelesítés).
 - E-mail cím megerősítése és elfelejtett jelszó visszaállítása (külső e-mail szolgáltatóval).
 - Felhasználói profil kezelése (név, életkor, nem, magasság, súly, test zsír százalék, heti aktivitási szint kiválasztása és a célsúly megadása).
- Aktivitás követés:
 - Új aktivitás (Activitycatalog) felvitele (aktivitás neve, kalória és a hozzá tartozó időegység).
 - Aktivitás listázása, szerkesztése és törlése.
- Ételek és Receptek kezelése:
 - Új alapanyagok (Food) felvitele tápérték adatokkal (étel neve és tömege grammban, kalória, makrók,).
 - Komplex receptek (Recipe) összeállítása több alapanyagból, ahol a rendszer automatikusan kiszámolja az összesített tápértéket.
 - Saját ételek és receptek listázása, szerkesztése és törlése.
- Naplózás (Daily Note):
 - Gyorskeresés az ételek és receptek között név alapján
 - Napi étkezések rögzítése étkezéstípusonként (pl. reggeli, ebéd).

- A kiválasztott ételek/receptek hozzáadása a naplóhoz a fogyasztott mennyiség megadásával.
- Aktuális napi testsúly rögzítése.
- A napi bevitel (kalória, makrók) valós idejű összesítése és vizuális összevetése a beállított céllal.
- Mozgásformák kiválasztása katalógusból és a hozzá tartozó időegység megadása
- Statisztika és Vizualizáció:
 - Naptár funkció a korábbi napok adatainak visszakeresésére.
 - A testsúly alakulásának grafikonos megjelenítése az idő függvényében.

3.2 Nem-funkcionális követelmények

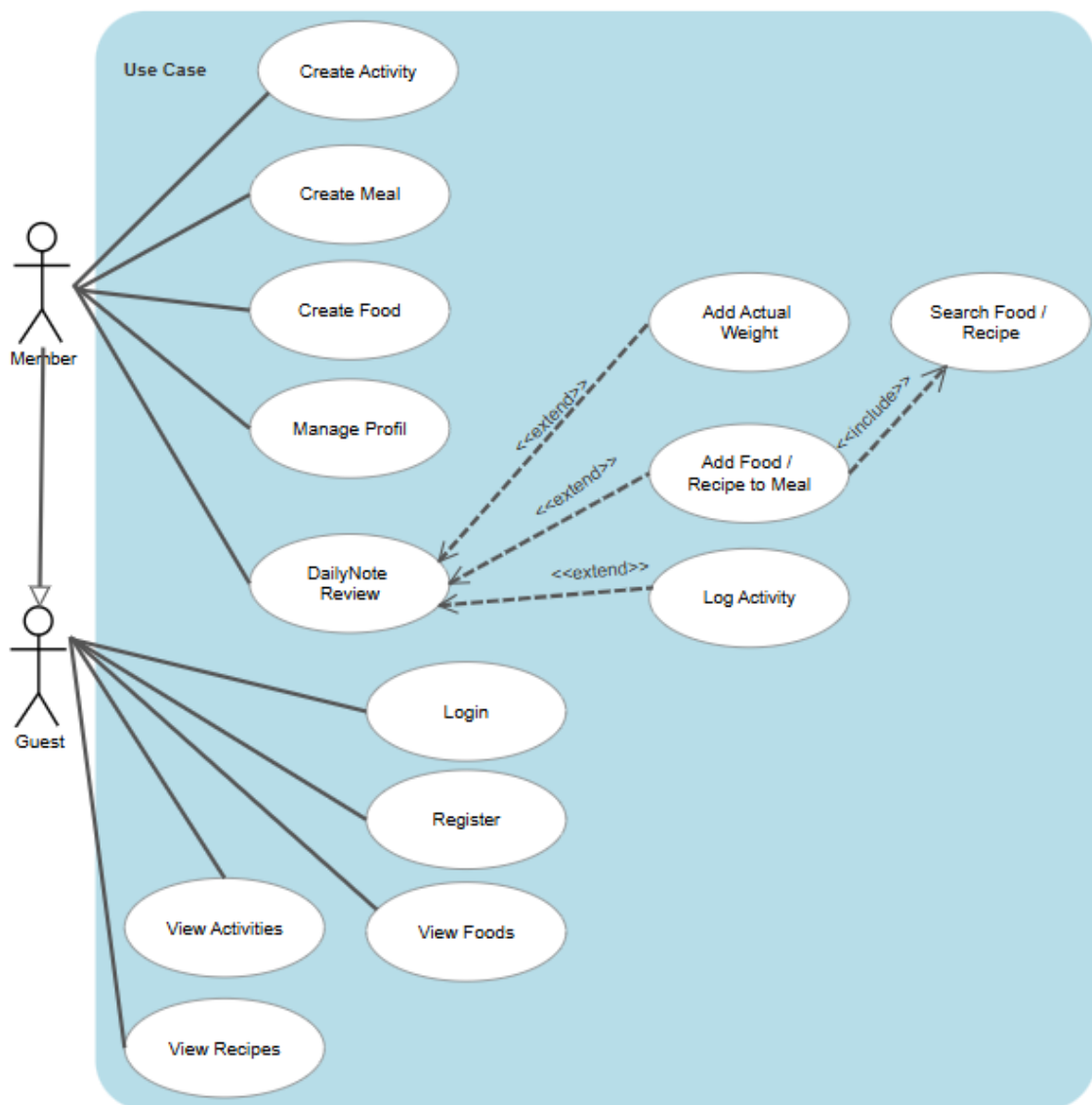
A rendszerrel szemben támasztott minőségi követelményeket az alábbiakban határoztam meg:

- Biztonság: A jelszavakat nem olvasható formában, hanem hash-elve tárolom az adatbázisban (ASP.NET Identity). A kommunikáció a kliens és a szerver között HTTPS csatornán zajlik, a végpontokat pedig hitelesítés védi.
- Teljesítmény: Az alkalmazásnak gyors válaszidőt kell biztosítania. Ennek érdekében aszinkron adatbázis-műveleteket és kliensoldali renderelést (SPA) alkalmaztam.
- Használhatóság: A felületnek reszponzívnak kell lennie, hogy asztali számítógépen és mobil eszközön is kényelmesen használható legyen. Ezt a Bootstrap keretrendszerrel biztosítottam.
- Skálázhatóság: A többretegű architektúra (Layered Architecture) lehetővé teszi a rendszer későbbi bővítését új funkciókkal anélkül, hogy a meglévő kódot újra kellene írni.

3.3 Szerepkörök és használati esetek

A rendszerben a jelenlegi megvalósításban két fő szerepkört különböztettem meg:

- Vendég (Guest): Nem regisztrált látogató. Jogosultsága korlátozott, a kezdőoldalt (Food), az Aktivitásoldalt (Activitycatalog), a Recepteket (Recipes), továbbá a regisztrációt és a bejelentkezést éri el.
- Regisztrált felhasználó (Member): Bejelentkezett felhasználó, aki hozzáfér a rendszer teljes funkcionalitásához: vezetheti a naplóját, létrehozhat saját ételeket és recepteket, továbbá sport tevékenységeket, valamint követheti a súlyának alakulását és menedzselheti a profilját a céljának megfelelően.



3. ábra Use Case diagram

A legfontosabb használati eset, a "Napi naplózás" folyamata a következőképpen néz ki: a felhasználó kiválasztja az adott napot, majd az étkezés típusát (pl. Ebéd). A kereső segítségével kiválaszt egy ételt vagy receptet, megadja a mennyiséget, és menti a

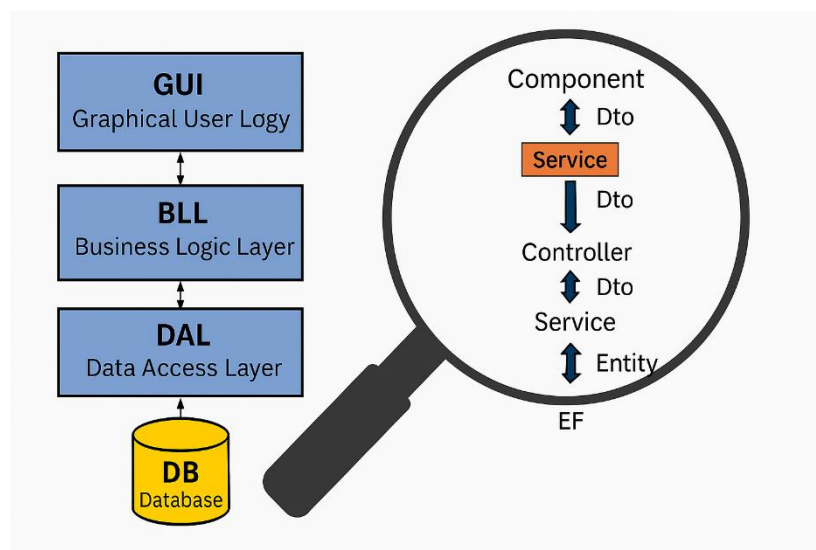
bejegyzést. A rendszer ezt követően azonnal frissíti a napi összesítő sávokat. Megadhatja ezen túlmenően a napi aktivitását és aznapi testsúlyát is a felhasználó.

3.4 Rendszerarchitektúra

A rendszert a modern webes fejlesztésben elterjedt kliens-szerver architektúrára építettem. A megoldás két fizikailag és logikailag is elkülönülő komponensből áll:

- **Frontend (Kliens):** Egy Angular alapú egyoldalas alkalmazás (SPA), amely a felhasználó böngészőjében fut. Feladata a felhasználói interakciók kezelése és az adatok megjelenítése.
- **Backend (Szerver):** Egy ASP.NET Core Web API, amely az üzleti logikát valósítja meg, és kapcsolatot tart az adatbázissal.
- **Külső szolgáltatások (3rd Party Services):** A regisztrációs folyamat és a biztonságos jelszókezelés támogatására a rendszer egy külső e-mail küldő szolgáltatást (Mailjet) integrál. Ez a komponens nem része a belső hálózatnak, a backend REST API hívásokon keresztül kommunikál vele a tranzakcionális e-mailek kiküldéséhez.

A két réteg között a kommunikáció REST (Representational State Transfer) alapú HTTP kérésekkel történik, JSON adatformátumban. Ez a laza csatolás biztosítja, hogy a kliens és a szerver egymástól függetlenül fejleszthető legyen.



4. ábra Rendszerarchitektúra diagram

3.5 Adatbázis-tervezés

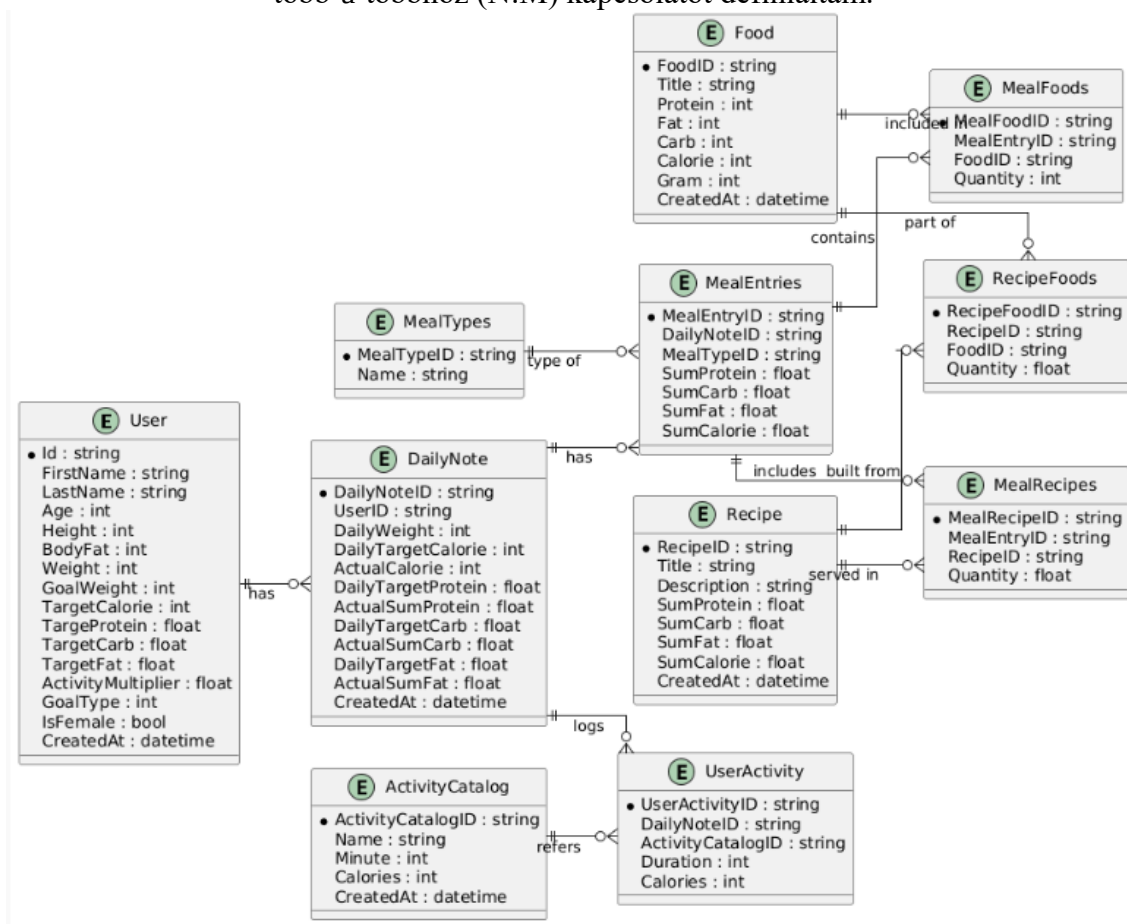
Az adatok tárolására relációs adatbázist terveztem (Microsoft SQL Server). A tervezés során a Code-First megközelítést alkalmaztam, azaz először a C# modellosztályokat (Models mappa) hoztam létre, és ebből generáltam le az adatbázis-sémát az Entity Framework Core segítségével.

A rendszer főbb entitásai és kapcsolatai:

- User: A központi entitás, amelyhez minden személyes adat (naplók, receptek) kapcsolódik.
- DailyNote (Napi napló): A naplózás gerince. A kapcsolatokat úgy alakítottam ki, hogy egy DailyNote-hoz tetszőleges számú étkezés (MealEntries) és sporttevékenység (UserActivity) tartozhasson (1:N kapcsolat). Ez teszi lehetővé, hogy a felhasználó egy napon belül több különböző mozgásformát (pl. futás, konditerem) is rögzíthessen.
- MealEntries (Étkezés): Az étkezéseket úgy modelleztem, hogy azok ne csak egyetlen elemet tartalmazhassanak. A MealFoods és MealRecipes kapcsolótáblák segítségével egyetlen étkezéshez (pl. Ebéd) több különböző étel és recept is hozzárendelhető.

Recipe és Food: A receptek és az ételek közötti kapcsolatot egy dedikált kapcsolótáblával (RecipeFoods) valósítottam meg. Mivel egy recept számos alapanyagból állhat, és egy alapanyag (pl. tojás) számtalan receptben szerepelhet, itt

több-a-többhöz (N:M) kapcsolatot definiáltam.



5. ábra A megvalósított rendszer Entitás-kapcsolat (ER) diagramja

3.6 Fejlesztési környezet és eszközök

A fejlesztés során a verziókezelést Git rendszerrel oldottam meg, a forráskódot pedig a GitHub felhőszolgáltatásban tároltam. Ez lehetővé tette a változások nyomon követését és a biztonsági mentést. A fejlesztői környezetként a Visual Studio (backend) és a Visual Studio Code (frontend) integrált fejlesztői környezeteket használtam, amelyek hatékony hibakeresési és kódkiegészítési funkciókkal támogatták a munkámat.

4 Önálló munka bemutatása

A diplomaterv legfontosabb részeként ebben a fejezetben ismertetem az általam fejlesztett webalkalmazás konkrét megvalósítását. Részletezem a tervezési döntéseimet, az adatbázis felépítését, valamint a szerver- és kliensoldali üzleti logika implementációját.

4.1 Rendszertervezés és Adatmodell

A fejlesztés első lépéseként az adatok szerkezetét és a rendszer alapvető működési logikáját definiáltam. Mivel az alkalmazás központi eleme a táplálkozási adatok és a felhasználói naplók pontos kezelése, elengedhetetlen volt egy robusztus és jól strukturált adatmodell kialakítása.

4.1.1 Adatbázis koncepció

Az adatok perzisztens tárolására Microsoft SQL Server adatbázis-kezelőt választottam. A döntésemet az indokolta, hogy ez a technológia natívan integrálható a .NET ökoszisztémába, és kiváló teljesítményt nyújt a relációs adatok kezelésében.

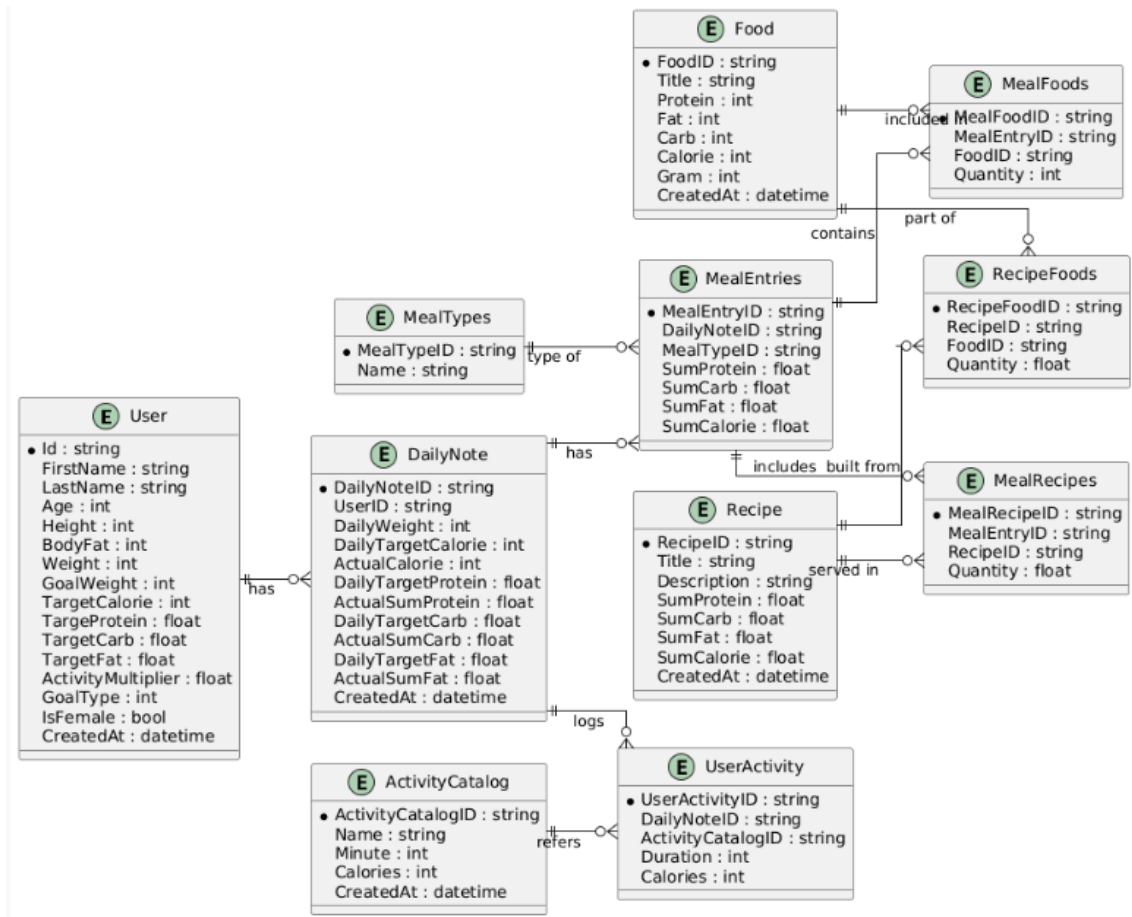
A fejlesztés során a Code-First megközelítést alkalmaztam az Entity Framework Core (EF Core) keretrendszer segítségével. Ez a módszer lehetővé tette számomra, hogy az adatbázis sémáját közvetlenül C# osztályokként (modellekként) definiáljam a kódban (HealthyAPI/Models), majd ezekből generáljam le az adatbázist.

Ez a megközelítés számos előnnyel járt a munkám során:

- Verziókövetés: Az adatbázis-séma változásait (Migrations) a kóddal együtt tudtam verzió kezelni a Git rendszerben.
- Típusbiztonság: A C# nyelv szigorú típusossága már a kódolás során biztosította az adatok konzisztenciáját.
- Gyorsaság: Nem kellett SQL szkripteket kézzel írnom, az EF Core automatikusan elvégezte a táblák és kapcsolatok létrehozását a DbContext konfigurációim alapján.

4.1.2 Az Adatmodell részletezése:

Az alkalmazás adatmodelljét úgy terveztem meg, hogy az támogassa a felhasználók, ételek, receptek és a napi tevékenységek közötti komplex kapcsolatokat. A rendszer entitás-kapcsolat (ER) diagramját a 4.1. ábra szemlélteti.



6. ábra A megvalósított rendszer Entitás-kapcsolat (ER) diagramja

A legfontosabb entitások és szerepük a rendszerben:

User (Felhasználó): Ez a központi entitás, amely kiterjeszti az ASP.NET Identity alapértelmezett felhasználói modelljét. A HealthyAPI/Models/User.cs fájlban olyan egyedi tulajdonságokat definiáltam, mint az Age (kor), Height (magasság), Weight (jelenlegi súly), BodyFat (Testzsír), IsFemale (milyen nemű), ActivityMultiplier (heti aktivitás szint) és a GoalWeight (célsúly). Itt tárolom a felhasználó napi makrotápanyag-céljait is (TargetCalorie, TargetProtein stb.), amelyeket a rendszer a regisztrációkor vagy a profil szerkesztésekor számol ki.

DailyNote (Napi napló): A naplózás gerince. Egy felhasználónak minden naphoz egyetlen DailyNote bejegyzése tartozhat. Ez az entitás tárolja az adott napra vonatkozó összesített értékeket (ActualCalorie, DailyWeight stb.) és a kapcsolatokat az étkezésekhez továbbá az aktivitáshoz.

MealEntries (Étkezés): A DailyNote alatt helyezkedik el, és az egyes napszakokat reprezentálja (pl. Reggeli, Ebéd). Egy naplóhoz több étkezés is tartozhat (1:N kapcsolat).

Food (Étel) és Recipe (Recept): Az ételeket a Food entitás, az ezekből összeállított fogásokat a Recipe entitás reprezentálja. Mindkettő tartalmazza a tápérték adatokat (fehérje, zsír, szénhidrát, kalória).

RecipeFoods (Recept-összetevők): Mivel egy recept több ételből állhat, és egy étel több receptben is szerepelhet, a Recipe és Food között N:M (sok-a-sokhoz) kapcsolat áll fenn. Ezt a RecipeFoods kapcsolótáblával oldottam meg, amely a mennyiséget (Quantity) is tárolja.

MealFoods és MealRecipes (Étkezés-tételek): Hasonlóan a receptekhez, az egyes étkezések (MealEntry) tartalmát is kapcsolótáblákkal valósítottam meg. A MealFoods az egyszerű alapanyagokat, míg a MealRecipes a kész recepteket rendeli hozzá az adott étkezéshez, tárolva az elfogyasztott mennyiséget is. A kapcsolatot egy-a-többhöz (1:N) viszonyként alakítottam ki, ami lehetővé teszi, hogy egyetlen étkezéshez (pl. Ebéd) tetszőleges számú ételt és receptet is rögzíthessen a felhasználó vegyesen (pl. egy kész recept mellé külön felvihet még egy gyümölcsöt is), a rendszer pedig ezeket egyetlen étkezésként kezeli és összegzi.

MealTypes (Étkezéstípusok): Ez az entitás definiálja a választható étkezési kategóriákat (pl. Reggeli, Ebéd, Vacsora). A rendszert úgy terveztem, hogy egy naphoz (DailyNote) több étkezés (MealEntry) is tartozhasson, amelyek mindegyike hivatkozik egy-egy étkezéstípusra. Ez teszi lehetővé, hogy a felhasználó elkülönítve, kategóriánként rögzíthesse és követhesse nyomon a napi tápanyagbevitelét.

UserActivity és ActivityCatalog (Aktivitások): A táplálkozás mellett a rendszer a sporttevékenységeket is kezeli. Az ActivityCatalog tartalmazza a választható mozgásformákat (és azok kalóriaégetési adatait). A UserActivity entitás kapcsolja össze ezeket a felhasználó napi naplójával (DailyNote). A kapcsolatot egy-a-többhöz (1:N) viszonyként modelleztem, ami biztosítja, hogy egy adott naphoz (DailyNote) tetszőleges

számú, különböző tevékenységet is rögzíthessen a felhasználó (pl. külön egy futást és egy úszást), a rendszer pedig ezeket külön-külön tárolja, de az elégetett kalóriákat napiszinten összesíti.

4.1.3 DTO-k szerepe

A fejlesztés során szigorúan elválasztottam az adatbázis-modelleket (amiket fentebb ismertettem) a külvilág felé kommunikált adatoktól. Erre a célra Data Transfer Object-eket (DTO) hoztam létre a HealthyAPI/DTOs mappában.

Ennek a rétegnek a bevezetése több okból is kritikus volt:

- **Adatvédelem:** Nem akartam minden adatot kiküldeni a kliensnek (pl. a felhasználó jelszavának hash-ét vagy belső azonosítókat). A UserDto például csak a megjelenítéshez szükséges adatokat (név, token) tartalmazza.
- **Cirkuláris hivatkozások elkerülése:** Az Entity Framework modelljei gyakran tartalmaznak körkörös hivatkozásokat (pl. a User hivatkozik a DailyNote-ra, ami visszahivatkozik a User-re). Ez a JSON serializálásnál hibát okozna. A DTO-k (pl. DailyNoteResponseDto) lapos szerkezetűek, így könnyen küldhetők hálózaton.
- **Kliens-specifikus formázás:** A DTO-kban lehetőségem volt az adatokat a frontend igényei szerint formázni (pl. dátumok átalakítása), anélkül, hogy az adatbázis szerkezetét módosítanom kellett volna.

A modellek és DTO-k közötti megfeleltetést (mapping) manuálisan végeztem a Service rétegben, hogy teljes kontrollom legyen az adatok áramlása felett.

4.2 A Backend megvalósítása (ASP.NET Core)

Az alkalmazás szerveroldali logikáját ASP.NET Core Web API segítségével valósítottam meg. A fejlesztés során a legfontosabb szempont a tiszta, karbantartható kód architektúra kialakítása és a biztonságos adatkezelés volt. Ebben az alfejezetben részletezem az architektúrális döntéseimet, a hitelesítési folyamatot, valamint a rendszer üzleti logikáját a profilkezeléstől kezdve a receptek összeállításán át a napi naplózásig.

4.2.1 Architektúra és Tervezési minták

A backend fejlesztése során a szoftverfejlesztésben elterjedt, többretegű (N-tier) architektúrát követtem. Különös figyelmet fordítottam a felelősségi körök szétválasztására (Separation of Concerns). Bár az Entity Framework Core használata miatt a klasszikus Repository réteget sok modern projektben elhagyják, én a Service (Szolgáltatás) réteget szigorúan alkalmaztam az üzleti logika és az adatbázis-elérés (Controller és DbContext) szétválasztására.

A rétegek közötti laza csatolást a Függőséginjektálás (Dependency Injection - DI) tervezési minta alkalmazásával értem el. A Program.cs fájlban regisztráltam az összes általam írt szolgáltatást és azok interfészeit az IoC (Inversion of Control) konténerbe. Megoldásomban a szolgáltatások regisztrációját Scoped élettartammal állítottam be, ami biztosítja, hogy minden HTTP kéréshez új példány jöjjön létre, ezzel optimalizálva az erőforrás-használatot.

```
builder.Services.AddScoped<IFoodService, FoodService>();
builder.Services.AddScoped<IPhotoService, PhotoService>();
builder.Services.AddScoped<IMealFoodsService, MealFoodsService>();
builder.Services.AddScoped<IMealTypeService, MealTypeService>();
builder.Services.AddScoped<IRecipeFoodService, RecipeFoodService>();
builder.Services.AddScoped<IRecipeService, RecipeService>();
builder.Services.AddScoped<IMealRecipesService, MealRecipesService>();
builder.Services.AddScoped<IMealEntriesService, MealEntriesService>();
builder.Services.AddScoped<IActivityCatalogService,
ActivityCatalogService>();
builder.Services.AddScoped<IUserActivityService, UserActivityService>();
builder.Services.AddScoped<IUserProfileService, UserProfileService>();
builder.Services.AddScoped<IDailyNoteService, DailyNoteService>();
```

Ez a megoldás lehetővé tette, hogy a Controllerek (pl. DailyNoteController) ne függjenek a szolgáltatások konkrét implementációjától, csak az absztrakcióktól (interfészekről). A vezérlők (Controllers) feladata kizárólag a HTTP kérések fogadása, a bemeneti adatok (DTO-k) validálása és a megfelelő szolgáltatás meghívása lett, míg a komplex számítási logika a Service rétegbe került.

4.2.2 Hitelesítés és biztonság

Mivel az alkalmazás érzékeny egészségügyi adatokat (súly, testzsír, étkezési szokások) kezel, a biztonságos hitelesítés kiemelt fontosságú volt. A rendszert ASP.NET Core Identity alapokon építettem fel, amelyet JWT (JSON Web Token) alapú tokenes hitelesítéssel egészítettem ki a stateless (állapotmentes) API kommunikáció érdekében.

ASP.NET Core Identity konfiguráció: A Program.cs fájlban definiáltam a felhasználókezelés szabályait. Az AddIdentityCore<User> szolgáltatás regisztrálásakor szigorú, de felhasználóbarát jelszóházi rendet állítottam be (pl. minimum 6 karakter hossz), és biztonsági okokból bekapcsoltam a SignIn.RequireConfirmedEmail = true opciót, ami megakadályozza a bejelentkezést, amíg a felhasználó nem igazolja vissza az e-mail címét. Az adattároláshoz az AddEntityFrameworkStores<Context>() metódussal kötöttem össze az Identity rendszert az SQL adatbázissal.

4.2.2.1 A regisztráció és bejelentkezés folyamata

A felhasználókezelést az AccountController valósítja meg. A folyamat során Data Transfer Object-eket (DTO) használok az adatminimalizálás és a típusbiztonság érdekében.

Regisztráció: A kliens egy RegisterDto-t küld (Név, Email, Jelszó). A rendszer létrehozza a felhasználót, majd azonnal elindít egy e-mail megerősítési folyamatot.

Bejelentkezés: A LoginDto fogadása után a SignInManager ellenőrzi a hitelesítő adatokat. Sikeres azonosítás esetén a válasz egy UserDto, amely már tartalmazza a generált JWT token is, így a kliens azonnal, további kérések nélkül hitelesített állapotba kerül.

4.2.2.2 E-mail szolgáltatás (Mailjet)

A regisztrációs és „elfelejtett jelszó” folyamatokhoz (ResetPasswordDto) külső e-mail küldő szolgáltatót, a Mailjet-et integráltam. Ezt a logikát az EmailService osztályba szerveztem ki, amelyet Scoped élettartammal regisztráltam a Program.cs-ben. A megvalósítás során a Mailjet.Client könyvtárat és a TransactionalEmailBuilder mintát alkalmaztam a levelek összeállítására. Ez lehetővé tette, hogy a HTML tartalmú e-maileket strukturáltan, C# kódból építsem fel, majd aszinkron módon (SendTransactionalEmailAsync) küldjem el a Mailjet API-nak JSON formátumban.

4.2.2.3 JWT Token generálás és Validáció

A sikeres bejelentkezés kulcsa a JWTService, amely felelős az aláírt tokenek létrehozásáért. A Program.cs-ben konfiguráltam a JwtBearer hitelesítést, ahol beállítottam a token érvényesítési paramétereit (Issuer Signing Key validálása, Issuer

ellenőrzése), biztosítva, hogy csak a saját szerverem által aláírt tokeneket fogadja el az API.

```
public string CreateJWT(User user)
{
    var userClaims = new List<Claim>
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id),
        new Claim(ClaimTypes.Email, user.UserName),
        new Claim(ClaimTypes.GivenName, user.FirstName),
        new Claim(ClaimTypes.Surname, user.LastName),
    };
    var credentials = new SigningCredentials(_jwtKey,
SecurityAlgorithms.HmacSha256Signature);
    var tokenDescriptor = new SecurityTokenDescriptor
    {
        Subject = new ClaimsIdentity(userClaims),
        Expires = DateTime.UtcNow.AddDays(int.Parse(_config["JWT:ExpiresInDays"])),
        SigningCredentials = credentials,
        Issuer = _config["JWT:Issuer"]
    };
    var tokenHandler = new JwtSecurityTokenHandler();
    var jwt = tokenHandler.CreateToken(tokenDescriptor);
    return tokenHandler.WriteToken(jwt);
}
```

A fenti kódban látható, hogy a tokenben szabványos Claim-ként (állításként) beégetem a felhasználó egyedi azonosítóját (NameId) és e-mail címét. Ez teszi lehetővé a szerver számára, hogy a későbbi kéréseknél adatbázis-lekérdezés nélkül, pusztán a token dekódolásával azonosítsa a felhasználót (User.FindFirst(ClaimTypes.NameIdentifier)), jelentősen csökkentve ezzel a terhelést.

4.2.3 Felhasználói profil és Célok kezelése

Az alkalmazás „agya” a felhasználói profil, amely alapján a rendszer a napi célokat kalkulálja. Ezt a komplex üzleti logikát a UserProfileService osztályban valósítottam meg.

A felhasználó a regisztrációkor vagy a profil szerkesztésekor megadja fizikai paramétereit (kor, nem, magasság, súly, cél súly) és aktivitási szintjét. A rendszer ezekből az adatokból automatikusan számolja újra a szükségleteket a CalculateMacronutrients privát metódus segítségével.

A számítás lépései a következők:

Alapanyagcsere (BMR) meghatározása: A nemzetközileg elismert Mifflin-St Jeor képletet alkalmaztam, amely a legpontosabb becslést adja a nyugalmi energiafelhasználásra. A képlet figyelembe veszi a nemek közötti eltérést is (férfiaknál +5, nőknél -161 korrekciós tényezővel).

Napi energiaszükséglet (TDEE): A kapott BMR értéket megszoroztam a felhasználó által választott aktivitási szorzóval (ActivityMultiplier), amely pl 1.2 (ülő életmód), 1.9 (extrém aktivitás) értékek lehetnek.

Célok szerinti korrekció: A felhasználó céljától (GoalType) függően módosítottam a kalóriakeretet: fogyás (Cutting) esetén 500 kalóriát vontam le, míg tömegnövelés (Bulking) esetén 500 kalóriát adtam hozzá a napi szükséglethez a fenntartható változás érdekében.

Makrotápanyagok felosztása: A kiszámolt kalóriakeretet egy fix, kiegyensúlyozott arányban (30% fehérje, 30% zsír, 40% szénhidrát) osztottam szét, majd ezeket átváltottam grammra (fehérje/szénhidrát: 4 kcal/g, zsír: 9 kcal/g).

4.2.4 Ételek

A rendszer alapkövei az ételek, amelyekből később a felhasználók étkezéseket és recepteket állíthatnak össze. Ezt az adatkezelési logikát a FoodController és FoodService osztályok együttműködésével valósítottam meg.

A felhasználó új étel felvételekor megadja az alapanyag nevét és pl 100 grammra vetített tápérték adatait (fehérje, zsír, szénhidrát, kalória). A rendszer ezekből a nyers adatokból egy többlépcsős folyamat során hozza létre a végleges adatbázis-bejegyzést.

Az adatfeldolgozás lépései a következők:

Adatfogadás és Validáció: A beérkező kérést (FoodCreateDto) a rendszer először formai ellenőrzésnek veti alá (pl. kötelező mezők megléte), biztosítva, hogy csak helyes formátumú adatok kerüljenek feldolgozásra.

Entitás összeállítása: A validált DTO adataiból a rendszer példányosítja a végleges Food entitást. Ebben a lépésben történik meg az üzleti objektum technikai mezőinek – például a létrehozás pontos idejét jelölő CreatedAt dátumbélyegnek – az automatikus beállítása is.

Perzisztencia: Az előkészített entitást a FoodService osztály aszinkron módon adja hozzá a DbContext-hez, majd a SaveChangesAsync metódussal véglegesíti a tranzakciót az SQL adatbázisban.

Válasz generálása: A sikeres mentést követően a rendszer egy FoodResponseDto objektumba csomagolva adja vissza a létrehozott elem adatait (beleértve a generált azonosítót is), így a kliensoldali felület azonnal, újratöltés nélkül képes megjeleníteni az új alapanyagot.

4.2.5 Sporttevékenységek kezelése

A rendszer nemcsak a bevitt, hanem az energiafelhasználást is nyomon követi. A sporttevékenységek rögzítését és kalkulációját a UserActivityService osztályban valósítottam meg.

A folyamat a felhasználó által megadott időtartam és a kiválasztott mozgásforma alapján zajlik:

Referenciaadatok lekérése: A Create metódus első lépéseként a rendszer lekéri a választott mozgásformához tartozó katalógusadatokat (ActivityCatalog), amely tartalmazza az egységnyi időre (pl. 60 perc) eső kalóriaégetést.

Kalóriakalkuláció: A rendszer automatikusan elvégzi a számítást: a katalógusértéket arányosítja a felhasználó által megadott időtartammal (Duration).

Integráció a naplóval: A mentést követően a szolgáltatás azonnal meghívja a DailyNoteService-t, hogy frissítse az adott naphoz tartozó összesített „elégetett kalória” mezőt, így a felhasználó valós időben látja az energiamérlegének változását.

4.2.6 Receptkezelés:

A rendszer egyik legösszetettebb algoritmus a receptek összeállítása és tápérték-számítása. A követelmény az volt, hogy a felhasználó szabadon válogathasson össze alapanyagokat (Food), a rendszer pedig ezekből egyetlen, egységes „Recept” entitást hozzon létre, automatikusan kalkulált tápértékekkel. Ezt a logikát a RecipesController és a RecipeService rétegekben implementáltam.

A folyamat vezérlése: A kérés a RecipesController Create végpontjára érkezik egy RecipeCreateDto formájában, amely tartalmazza a recept nevét, leírását, valamint a

kiválasztott összetevők listáját és mennyiségeit. A vezérlő a kérést közvetlenül továbbítja az üzleti logikai rétegnek.

Üzleti logika és Számítás (RecipeService): A komplex feldolgozást a RecipeService osztály Create metódusában valósítottam meg. A metódus az alábbi lépéseket hajtja végre:

Entitás előkészítése: Példányosítom az új Recipe objektumot, és beállítom az alapvető adatait (Cím, Leírás, Létrehozás dátuma).

Iteráció és Adatgyűjtés: Egy ciklus segítségével végigiterálok a kapott összetevőkön (Ingredients). Minden egyes tételnél lekérdezem az adatbázisból a hozzá tartozó Food entitást, hogy hozzáférjek a hiteles tápértékadatokhoz.

```
private async Task RecalculateNutrition(Recipe recipe)
{
    var recipeFoods = await _context.RecipeFoods.Include(rf => rf.Food)
        .Where(rf => rf.RecipeID == recipe.RecipeID).ToListAsync();

    recipe.SumProtein = recipeFoods.Sum(rf => rf.Quantity / 100 *
rf.Food.Protein);
    recipe.SumCarb = recipeFoods.Sum(rf => rf.Quantity / 100 *
rf.Food.Carb);
    recipe.SumFat = recipeFoods.Sum(rf => rf.Quantity / 100 * rf.Food.Fat);
    recipe.SumCalorie = recipeFoods.Sum(rf => rf.Quantity / 100 *
rf.Food.Calorie);
}
```

Valós idejű aggregálás: A ciklus belsejében végzem el a matematikai számításokat: az alapanyagok 100 grammra vetített értékeit a felhasználó által megadott mennyiséggel (Quantity) súlyozom, majd ezeket folyamatosan hozzáadom a recept összesített makróihoz (fehérje, zsír, szénhidrát, kalória).

Kapcsolatok mentése: A számítás mellett minden lépésben létrehozok egy RecipeFoods kapcsoló-rekordot is, amely rögzíti, hogy az adott recepthoz melyik ételből mennyi tartozik.

Tranzakció: Végezetül a kész Recipe entitást (a hozzárendelt RecipeFoods listával együtt) egyetlen tranzakcióban mentem el az adatbázisba a SaveChangesAsync hívással. Ez a megoldás leveszi a terhet a felhasználó válláról: a rendszer automatikusan és precízen végzi el a makrotápanyagok összesítését a kiválasztott alapanyagok alapján.

4.2.7 Üzleti logika - A Naplózás magja

Az alkalmazás központi funkciója a napi táplálkozás és aktivitás nyomon követése. Ezt a komplex logikát nem egyetlen monolitikus osztályban, hanem több, egymásra épülő szolgáltatás (DailyNoteService, MealEntriesService, MealFoodsService) együttműködésével valósítottam meg. Ez a felépítés biztosítja a rendszer modularitását és a felelősségi körök tiszta szétválasztását.

4.2.7.1 A DailyNote életciklusa (Inicializálás)

A naplózási folyamat alapja, hogy létezzen az adott napra vonatkozó keretrendszer. Ezt a DailyNoteService vezérli. Amikor a felhasználó belép a napló felületre, a rendszer ellenőrzi a mai bejegyzés meglétét. Hiány esetén a CreateDailyNote metódus automatikusan létrehozza azt, és ami a legfontosabb: ebben a pillanatban rögzíti a felhasználó aktuális céljait (a UserProfile-ből átmásolja a TargetCalorie, TargetProtein stb. értékeket). Ez a tervezési döntés biztosítja az adatbiztonságot: ha a felhasználó később módosítja a profilját, a múltbeli naplói nem torzulnak, azok megőrzik az akkori célértékeket.

4.2.7.2 Étkezések hierarchiája (MealEntries)

A naplón belüli étkezések (pl. Reggeli, Ebéd) kezelését a MealEntriesService végzi. A rendszert úgy terveztem, hogy egy DailyNote tetszőleges számú étkezést tartalmazhat, amelyeket a MealType törzsadatok (pl. "Reggeli", "Vacsora") alapján kategorizálok. Ez a réteg felel az adott napszak összesített tápértékeinek (SumCalorie, SumProtein) tárolásáért, amelyeket a rendszer valós időben frissít, amint a felhasználó új tételt ad hozzá.

4.2.7.3 Tételek rögzítése és a Számítási lánc (Recalculation Chain)

A legalsó szinten a konkrét ételek és receptek hozzáadása történik, amit a MealFoodsService és MealRecipesService osztályok kezelnek. A legnagyobb kihívást itt az adatok konzisztenciájának megőrzése jelentette minden egyes módosítás után.

Erre egy többlépcsős újraszámolási láncot (Recalculation Chain) implementáltam:

Tétel rögzítése: A felhasználó hozzáad egy ételt (pl. 200g csirkemell). A MealFoodsService elmenti a tételt.

Étkezés szintű frissítés: A szolgáltatás azonnal lekéri az érintett étkezéshez (MealEntry) tartozó összes ételt és receptet, majd újrakalkulálja az étkezés aggregált tápértékeit (RecalculateMealEntryNutrition).

Napló szintű frissítés: A lánc végén a rendszer meghívja a DailyNoteService-t, amely összegzi az összes étkezés adatait, és frissíti a napi főösszegeket.

Ez a hierarchikus működés garantálja, hogy a felhasználó a felületen (pl. a folyamatsávokon) mindig matematikai pontossággal a helyes, aktuális értékeket lássa, függetlenül attól, hogy egyszerű ételt vagy komplex receptet rögzített.

4.2.7.4 Testsúly naplózása

A testsúlyváltozás követése kritikus funkció a célok elérésében. Ezt a logikát a DailyNoteService UpdateWeight metódusában implementáltam. A rendszer az aktuális napi naplóbejegyzésben (DailyNote) tárolja el a mért súlyt. Ez teszi lehetővé a későbbi történeti visszatekintést és a grafikonos ábrázolást, továbbá a következő napi testsúly az itt megadott adattal fog inicializálódni.

4.2.8 Adatszolgáltatás a vizualizációhoz (Grafikon és Naptár)

A frontend oldali vizualizáció (súlygrafikon, naptárnézet) kiszolgálása érdekében a backend specifikus végpontokat biztosít, amelyek aggregált adatokat szolgáltatnak.

- Naptárnézet: A felhasználó itt képes kiválasztani mely napi bejegyzését szeretné megtekinteni, ezért pedig a GetByDate metódus felel.
- Súlygrafikon: A GetWeightHistoryAsync metódus időrendben adja vissza a rögzített testsúly-adatokat. Mivel a DailyNote tábla tartalmazza a DailyWeight mezőt és a CreatedAt dátumot, egy egyszerű LINQ lekérdezéssel kinyerhető a felhasználó súlyváltozásának teljes története, amit a frontend közvetlenül a diagram kirajzolásához használ.

4.2.9 API Dokumentáció:

A modern webfejlesztésben elengedhetetlen az API megfelelő dokumentálása. Erre a célra a Swagger/OpenAPI szabványt használtam, amelyet az NSwag eszköztárral egészítettem ki.

A Program.cs-ben konfiguráltam a Swagger generátort, amely futásidőben feltérképezi a Controllereket és a DTO-kat, majd létrehoz egy interaktív dokumentációs felületet (/swagger).

Ennek a megoldásnak a legnagyobb előnye a frontend fejlesztés során jelentkezett. Az NSwag stúdió segítségével (amelynek konfigurációját az ad.nswag fájlban tároltam) egyetlen kattintással képes voltam legenerálni a teljes TypeScript kliens kódot (NswagGenerated.ts). Ez azt jelentette, hogy a frontend oldalon nem kellett manuálisan megírnom a HTTP hívásokat és a modelleket, hanem egy típusos, autokomplettel támogatott szolgáltatást kaptam, ami drasztikusan csökkentette a hibalehetőségeket és felgyorsította a munkát.

4.3 A Frontend megvalósítása

A felhasználói felületet Angular 19 keretrendszer segítségével, modern egyoldalas alkalmazásként (SPA) valósítottam meg. A kliens oldali fejlesztés során a fő célom egy reszponzív, gyors és a backenddel szorosan integrált felület kialakítása volt. Ebben az alfejezetben bemutatom a kliens architektúráját, a backenddel való kommunikáció módját, valamint a legösszetettebb komponensek belső logikáját.

4.3.1 Kliens oldali architektúra és Modularitás

Az Angular alkalmazásomat tudatosan moduláris architektúrára terveztem, hogy a kód átlátható, karbantartható és logikusan strukturált legyen. A fejlesztés során a „Separation of Concerns” (Felelősségi körök szétválasztása) elvét követtem, így az alkalmazás különböző funkcionális egységeit dedikált könyvtárakba és modulokba szerveztem a src/app mappán belül.

4.3.1.1 Moduláris felépítés

A monolitikus struktúra helyett (ahol minden egyetlen modulban van) három fő modulra bontottam az alkalmazást:

AppModule (Fő belépési pont): Ez a gyökérmodul (app.module.ts), amely összefogja az alkalmazást. Itt importálom a globális függőségeket (pl. BrowserModule, HttpClientModule) és itt bootstrappelem a fő komponenst (AppComponent). Ez a modul felelős az alkalmazás elindításáért és az alapvető navigáció kezeléséért.

AccountModule (Felhasználókezelés): A felhasználói azonosítással és fiókkezeléssel kapcsolatos összes logikát egy dedikált modulba (AccountModule) szerveztem ki. Ide tartoznak a bejelentkezés (LoginComponent), a regisztráció (RegisterComponent), valamint a jelszó-visszaállítás komponensei. Ezzel a megoldással elértem, hogy a hitelesítési logika fizikailag is elkülönüljön az alkalmazás üzleti funkcióitól, ami átláthatóbbá tette a kódbázist.

SharedModule (Megosztott erőforrások): A kódDuplikáció elkerülése (DRY - Don't Repeat Yourself elv) érdekében létrehoztam egy SharedModule-t. Ide gyűjtöttem minden olyan komponenst, amelyet az alkalmazás több pontján is felhasználok.

- Itt deklaráltam a ValidationMessagesComponent-et, amely egységesen jeleníti meg az űrlaphibákat.
- Ide került a NotificationComponent is, amely a felugró visszajelzéseket kezeli. Ennek a modulnak a használata lehetővé tette, hogy ezeket a komponenseket bárhol egyszerűen beilleszthessem, anélkül, hogy újra kellene definiálnom őket.

4.3.1.2 Könyvtárstruktúra

A fájlrendszer szintjén is szétválasztottam az üzleti és a technikai logikát:

- **pages könyvtár:** Ide helyeztem az alkalmazás "érdemi" oldalait, ahol a felhasználó a napi tevékenységét végzi. Itt találhatóak a naplózás (daily-note), a receptkezelés (recipes, create-recipes) és a profilkezelés (create-profile) komponensei. Ezek alkotják az üzleti logikai réteget.
- **shared könyvtár:** Ez tartalmazza a technikai infrastruktúrát: a modelleket (DTO-k), a Guard-okat (útvonalvédelem), az Interceptorokat (token kezelés) és az újrafelhasználható UI elemeket.

4.3.1.3 Az alkalmazás kerete (Layout)

A felhasználói felület vázát és az alkalmazás belépési pontját az AppComponent adja. Ennek implementálásakor három fő technikai kihívást kellett megoldanom: a responzív, mindig az oldal aljára tapadó láblécet (Footer), a navigáció reaktív állapotkezelését, valamint a felhasználói munkamenet (session) automatikus helyreállítását az oldal újratöltésekor.

Layout technikai megvalósítása (Footer): A webfejlesztésben gyakori probléma, hogy kevés tartalom esetén a lábléc "felcsúszik" a képernyő közepére. Ezt a problémát modern CSS Flexbox technikával oldottam meg az `app.component.html` fájlban. A body elemet flexibilis konténerként definiáltam, amelynek minimális magasságát a képernyő teljes magasságára (`min-height: 100vh`) állítottam, az elemek irányát pedig oszloposra (`flex-direction: column`).

A struktúra három fő elemből áll:

- `app-navbar`: A felső navigációs sáv fix magassággal.
- Tartalomtároló (`div.container`): Ennek az elemnek a `flex: 1` stílust adtam. Ez a beállítás arra utasítja a böngészőt, hogy ez az elem töltsse ki az összes rendelkezésre álló szabad helyet a fejléc és a lábléc között. Így, ha a tartalom kevés, a konténer akkor is "kinyúlik", és a láblécet a képernyő aljára szorítja.
- `app-footer`: A lábléc, amely így mindig vizuálisan a helyén marad. A `FooterComponent`-ben dinamikus dátumkezelést is implementáltam (`new Date().getFullYear()`), hogy a copyright évszám mindig aktuális legyen.

Reaktív Navigáció és Menükezelés: A menüpontok láthatóságát (pl. "Login" vs. "Logout") nem manuális változókkal, hanem reaktív módon, az Angular `AsyncPipe` (`| async`) segítségével vezérelem. A nézet közvetlenül feliratkozik az `AccountService`-ben található `user$ observable`-re.

- Ha az érték null, a rendszer a vendég-menüt (Bejelentkezés, Regisztráció) rendereli ki.
- Ha van érvényes `User` objektum, a rendszer a bejelentkezett nézetet mutatja, megjelenítve a felhasználó keresztnévét és a "Logout" gombot.

A felhasználói élmény javítása érdekében a `routerLinkActive="active"` direktívát használtam. Ez automatikusan hozzáadja az `active` CSS osztályt ahhoz a menüponthoz, amelyen a felhasználó éppen tartózkodik, vizuális visszajelzést adva a navigációról.

Alkalmazás inicializálása és Token-ellenőrzés: Az egyoldalas alkalmazások kritikus pontja, hogy az oldal frissítésekor (F5) ne vesszen el a bejelentkezett állapot. Ezt a logikát az `AppComponent` TypeScript kódjában, az `ngOnInit` életciklusban valósítottam meg a `refreshUser()` módszerrel.

A folyamat a következő:

- Az alkalmazás indulásakor a rendszer ellenőrzi, van-e elmentett JWT token a böngésző localStorage-ában (`this.accountService.getJWT()`).
- Ha talál tokenet, egy API hívást indít a backend felé (`refreshUser`), hogy ellenőrizze annak érvényességét és lekérje a friss felhasználói adatokat.
- Hibakezelés: Külön figyelmet fordítottam a lejárt tokenek kezelésére. A `subscribe` blokk `error` ágában figyelem a 401 `Unauthorized` státuszkódot. Ha a szerver elutasítja a tokenet (pl. lejárt), a kliens automatikusan kijelentkezteti a felhasználót (`logout`), és a `SharedService`-en keresztül egy felugró értesítésben ("Account blocked") tájékoztatja a hiba okáról.

4.3.2 Navigáció és Útvonal-védelem (Routing & Security)

Egy egyoldalas alkalmazásnál (SPA) kritikus fontosságú a kliens oldali útvonalválasztás (Routing) helyes konfigurálása, hiszen ez határozza meg, hogy a felhasználó milyen tartalmat lát az URL alapján, anélkül, hogy az oldal újratöltődne. Ezt a logikát az Angular beépített `RouterModule`-jára építve, az `app-routing.module.ts` fájlban definiáltam.

4.3.2.1 Útvonalak definíciója és Hierarchiája

A routes tömb összeállításakor logikailag több fő csoportba rendeztem az elérési útvonalakat, amelyek különböző hozzáférési szinteket és technikai megoldásokat igényeltek.

Publikus útvonalak (Public Routes): A tömb elején definiáltam azokat az oldalakat, amelyek bejelentkezés nélkül is elérhetők bárki számára. Úgy terveztem a rendszert, hogy a látogatók regisztráció nélkül is megtekinthessék az adatbázis tartalmát (katalógusokat), de módosítani ne tudjanak.

- `path: 'food'`: Az ételek listázása (`FoodComponent`).
- `path: 'activitycatalog'`: A sporttevékenységek katalógusa (`ActivityCatalogComponent`).
- `path: 'recipes'`: A receptek listája (`RecipesComponent`).

Beállítottam egy alapértelmezett átirányítást is: ha a felhasználó az üres gyökér útvonalra (path: "") érkezik, a rendszer automatikusan a /food oldalra navigálja (redirectTo: 'food'), így a belépéskor azonnal releváns tartalmat lát.

Védett útvonalak és a Gyermek-útvonalak (Children): Az alkalmazásom egyik legfontosabb architektúrális döntése a védett útvonalak csoportosítása volt. Ahelyett, hogy minden egyes szerkesztő felülethez (Create/Edit) külön-külön rendeltem volna hozzá a jogosultság-ellenőrzést, egyetlen közös szülő útvonalat hoztam létre.

Ezt a szülő útvonalat üres útvonalként (path: "") definiáltam, és ehhez rendeltem hozzá a canActivate: [AuthorizationGuard] védelmet. A tényleges funkcionális oldalakat ennek a szülőnek a children (gyermek) tömbjében helyeztem el.

```
{ path: '',
  runGuardsAndResolvers: 'always',
  canActivate: [AuthorizationGuard],
  children: [
    { path: 'activitycatalog/create', component: CreateActivityCatalogComponent },
    { path: 'activitycatalog/edit/:id', component: CreateActivityCatalogComponent },
    { path: 'food/create', component: CreateFoodComponent },
    { path: 'food/edit/:id', component: CreateFoodComponent },
    { path: 'recipes/create', component: CreateRecipesComponent },
    { path: 'recipes/edit/:id', component: CreateRecipesComponent },
    { path: 'create-profile', component: CreateProfileComponent },
    { path: 'dailynote', component: DailyNoteComponent }, ], ], }
```

Ennek a hierarchikus felépítésnek kettős előnye van:

- Öröklődés: A children tömbben lévő összes útvonal (pl. /dailynote, /recipes/create) automatikusan öröklí a szülőre tett Guard-ot. Így garantált, hogy a naplóhoz vagy a profil szerkesztéséhez (create-profile) csak bejelentkezett felhasználó férhet hozzá.
- Dinamikus paraméterek: A szerkesztő útvonalaknál (pl. recipes/edit/:id) útvonal-paramétereket (:id) alkalmaztam. Ezt az azonosítót a komponensekben az ActivatedRoute szolgáltatás segítségével olvasom ki, hogy a backendről betölthessem a szerkesztendő elem adatait.

Lusta betöltés (Lazy Loading): Az alkalmazás indulási idejének optimalizálása érdekében a felhasználókezelést (AccountModule) leválasztottam a fő csomagról. A path: 'account' útvonalnál a loadChildren szintaxist alkalmaztam. Ez azt eredményezi, hogy a bejelentkezéshez és regisztrációhoz szükséges JavaScript kód nem töltődik le az első

oldalbetöltéskor, hanem csak akkor, amikor a felhasználó ténylegesen rákattint a "Login" gombra.

Végezetül a hibás URL-ek kezelésére a „wildcard” (**) útvonalat használtam, amely minden, a fentiekben nem definiált kérést a NotFoundComponent-re (404-es hibaoldal) irányít át.

4.3.2.2 Útvonal-védelem (AuthGuard) megvalósítása

A jogosulatlan hozzáférések megakadályozására implementáltam az AuthorizationGuard osztályt (shared/guards/authorization.guard.ts), amely a CanActivate interfészt valósítja meg.

A védelmi logika a következő lépésekből áll:

- A Guard feliratkozik az AccountService-ben található user\$ observable-re, amely a felhasználó aktuális állapotát (be van-e jelentkezve) reprezentálja.
- A map operátor segítségével megvizsgálom a user objektumot:
- Ha létezik (nem null): A felhasználó hitelesítve van, a Guard true értékkel tér vissza, engedélyezve a navigációt.
- Ha nem létezik: A rendszer azonnal megakadályozza a továbbhaladást (false). Ezzel egyidőben a SharedService segítségével egy felugró értesítésben ("Restricted Area") tájékoztatom a felhasználót, és átirányítom a bejelentkezési oldalra.

4.3.3 Kommunikáció a Backenddel (API Layer)

A modern webalkalmazások egyik legkritikusabb pontja a kliens és a szerver közötti adatcsere. A fejlesztés során a fő célkitűzésem az volt, hogy ez a kommunikáció típusbiztos, automatizált és könnyen karbantartható legyen, minimalizálva az emberi hiba lehetőségét.

4.3.3.1 Automatizált kliensgenerálás (NSwag)

A fejlesztés során a hagyományos, manuális megközelítés helyett (ahol minden HTTP kérést kézzel írunk meg) az automatizációt választottam. A backend oldalon konfigurált OpenAPI/Swagger specifikációt használtam fel arra, hogy az NSwag eszköztár segítségével legeneráljam a teljes kliens oldali adatelérési réteget.

A folyamatot az `ad.nswag` konfigurációs fájl vezérli. Ez az eszköz a backend futása közben elérhető `swagger.json` leírófájl alapján egyetlen, robusztus TypeScript fájlt (`src/app/shared/models/Nswag generated/NswagGenerated.ts`) hoz létre, amely több mint 6000 sornyi, automatikusan karbantartott kódot tartalmaz.

Generált Szolgáltatás Kliensek (Service Clients): Az NSwag minden egyes backend kontrollerhez létrehozott egy megfelelő Angular Injectable szolgáltatást. Ezek az osztályok tartalmazzák a végpontok hívásához szükséges teljes logikát (URL összeállítás, HTTP metódusok, hibakezelés). A projektben számos kliensosztály jött létre, amelyek közül a legfontosabbak:

- **AccountClient:** A felhasználókezelésért felel (pl. login, register, confirmEmail metódusok), közvetlen kapcsolatot biztosítva az AccountController-rel.
- **DailyNoteClient:** A naplózás központi eleme. Metódusai (pl. getDailyNote, addFoodToMeal) teszik lehetővé a napi bejegyzések lekérdezését és módosítását.
- **RecipesClient és FoodClient:** Az ételek és receptek kezelését végzik (pl. createRecipe, searchFood), biztosítva a CRUD műveleteket.
- **UserActivityClient és ActivityCatalogClient:** A sporttevékenységek naplózását és a katalógus elérését támogató szolgáltatások.
- **MealEntriesClient, MealFoodsClient:** A napló részleteinek (étkezések, tételek) finomhangolását végző kliensek.

Ezeket a klienseket a komponenseimben (pl. DailyNoteComponent) függőséginjektálással (DI) érem el, így a kódom tiszta és típusos marad.

Típusos DTO-k (Interfészek): A kliensfájl nemcsak a logikát, hanem a teljes adatmodellt is tartalmazza TypeScript interfészek formájában. Ez garantálja, hogy a frontend és a backend "egy nyelvet beszéljen". A generált modellek között megtalálhatóak:

Válasz modellek (Response DTOs): Mint például a `DailyNoteResponseDto`, `UserProfileResponseDto` vagy `RecipeResponseDto`. Ezek pontosan definiálják, milyen adatokat kap a kliens a szervertől (pl. a `DailyNoteResponseDto` tartalmazza a `dailyTargetCalorie` és `actualCalorie` mezőket).

Létrehozási modellek (Create DTOs): Az adatok küldésekor használt struktúrák, mint a `RecipeCreateDto` vagy `UserActivityCreateDto`. Ezek biztosítják, hogy csak valid struktúrájú adatot küldhessek a szervernek.

Segédtypusok: Olyan enyhébb típusok és interfészek (pl. `MealTypeResponseDto`, `FoodResponseDto`), amelyek a komplexebb objektumok építőkövei.

4.3.3.2 Központosított Token kezelés (`HttpInterceptor`)

A védett API végpontok eléréséhez minden egyes HTTP kérés fejlécében (Header) küldeni kell az érvényes JWT token. A DRY (Don't Repeat Yourself) elvét követve ezt a logikát nem a komponensekbe építettem be, hanem egy `HttpInterceptor` segítségével központosítottam.

A `src/app/shared/interceptors/jwt.interceptor.ts` fájlban implementáltam a `JwtInterceptor` osztályt, amely implementálja az Angular `HttpInterceptor` interfészét. Ez a technológia "middleware-ként" működik a kliens oldalon: minden kifelé menő kérést elfog, módosít, majd továbbít.

A megvalósított logika lépései:

- **Feliratkozás:** Az `intercept` metódusban feliratkozom az `AccountService`-ben tárolt `user$` observable-re, de a `take(1)` operátorral biztosítom, hogy csak az aktuális értéket vegye ki, és azonnal le is iratkozzon (így elkerülhető a memóriaszivárgás).
- **Ellenőrzés:** Megvizsgálom, hogy a felhasználó be van-e jelentkezve (létezik-e a `user` objektum és a hozzá tartozó `jwt` token).
- **Klónozás és Módosítás:** Mivel az Angularban a `HttpRequest` objektumok immutábilisak (nem módosíthatók), a kérést a `.clone()` metódussal lemásolom. A másolat fejlécéhez adom hozzá az `Authorization` kulcsot a `Bearer <token>` értékkel.
- **Továbbítás:** A módosított (klónozott) kérést adom át a `next.handle()` metódusnak, amely ténylegesen elküldi azt a szerver felé.

```

    intercept(request:      HttpRequest<unknown>,      next:      HttpHandler):
Observable<HttpEvent<unknown>> {
    this.accountService.user$.pipe(take(1)).subscribe({
      next: user => {
        if (user) {
          request = request.clone({
            setHeaders: {
              Authorization: `Bearer ${user.jwt}`
            }
          });
        }
      }
    });
  }
}
}
}
return next.handle(request);
}
}
}

```

4.3.4 Felhasználókezelés (Account Module)

A felhasználói fiókok kezelését, a hitelesítést és a biztonsági funkciókat egy dedikált modulban, az AccountModule-ban valósítottam meg. Ez a modul fizikailag is elkülönül az alkalmazás többi részétől (src/app/account), és saját útvonalválasztóval (AccountRoutingModule) rendelkezik, amely a lusta betöltés (Lazy Loading) révén csak szükség esetén töltődik be a böngészőbe.

4.3.4.1 Reaktív Állapotkezelés (AccountService)

A felhasználókezelés "agya" az AccountService osztály. Itt valósítottam meg azt a logikát, amely biztosítja, hogy az alkalmazás mindig tisztában legyen azzal, ki van bejelentkezve.

A legnagyobb kihívást az jelentette, hogy az Angular komponensek (pl. a Navbar) aszinkron módon töltődnek be, így biztosítanom kellett, hogy a bejelentkezési állapot bárki számára, bármikor elérhető legyen. Erre a célra egy ReplaySubject<User | null>(1) típusú objektumot (userSource) alkalmaztam.

Miért ReplaySubject: Szemben a hagyományos Subject vagy BehaviorSubject megoldásokkal, a ReplaySubject(1) eltárolja ("visszajátssza") az utolsó kibocsátott értéket az új feliratkozóknak. Így, ha az AuthGuard csak később iratkozik fel, akkor is azonnal megkapja az aktuális felhasználói objektumot, elkerülve a versenyhelyzeteket (Race Conditions).

Az oldal újratöltésekor (F5) a munkamenet folytonosságát a localStorage-ba mentett adatokkal és a refreshUser metódussal biztosítom, amely automatikusan megújítja az állapotot, ha talál érvényes token.

4.3.4.2 Regisztráció és Bejelentkezés

A beviteli űrlapok kezeléséhez minden esetben az Angular Reactive Forms modulját használtam, mivel ez lehetőséget ad a komplex validációs logika (pl. jelszóerősség) TypeScript oldali implementálására és tesztelésére.

Regisztráció (RegisterComponent): Az űrlap (registerForm) validátorai biztosítják, hogy a név minimum 3 karakter legyen, az e-mail cím megfeleljen a szabványos regex mintának, a jelszó pedig 6-15 karakter hosszú legyen. A "Create Account" gomb megnyomásakor a komponens feliratkozik az AccountService.register() metódusára. Siker esetén a SharedService segítségével értesítem a felhasználót, majd a bejelentkezési oldalra navigálok.

Bejelentkezés (LoginComponent): A LoginComponent nemcsak a hitelesítést végzi, hanem intelligensen kezeli a felhasználói utat és a hibákat is.

- Intelligens átirányítás: A komponens inicializálásakor (ngOnInit) kiolvasom a returnUrl paramétert az URL-ből. Ha a felhasználó korábban egy védett oldalról (pl. /dailynote) lett átirányítva a bejelentkezéshez, sikeres belépés után a rendszer automatikusan visszavigálja oda.
- Email megerősítés kezelése: A HTML sablonban külön figyelmet fordítottam arra az esetre, ha a bejelentkezés azért hiúsul meg, mert a felhasználó még nem erősítette meg az e-mail címét. Ha a hibaüzenet tartalmazza a "Please confirm your email" szöveget, dinamikus megjelenítek egy linket ("Click here to resend email confirmation link..."), amely meghívja a resendEmailConfirmationLink() metódust.

4.3.4.3 Elfelejtett jelszó és E-mail megerősítés

A fiókkezelés teljes életciklusát lefedtem az alábbi komponensekkel:

ConfirmEmailComponent: Ez a komponens végzi a regisztráció véglegesítését. Az ngOnInit életciklusban feliratkozom az ActivatedRoute paramétereire, és kinyerem az URL-ből érkező hitelesítő token és e-mail címet.

- Ezekkel az adatokkal automatikusan meghívom az `AccountService.confirmEmail` metódust.
- A visszajelzést egy `success` logikai változó vezérli: siker esetén (`success = true`) a felhasználó egy "Login" gombot lát, hiba esetén (pl. lejárt token) pedig a rendszer felajánlja a megerősítő link újraküldését a `resendEmailConfirmationLink()` metódus segítségével.

`SendEmailComponent` (Többfunkciós űrlap): A kódDuplikáció elkerülése érdekében egyetlen komponenst készítettem az "Elfelejtett jelszó" és az "E-mail megerősítés újraküldése" funkciókhoz. A komponens az URL paraméter (mode) alapján dönti el, melyik üzemmódban működjön.

`ResetPasswordComponent`: Ez a felület fogadja a felhasználót, amikor az e-mailben kapott jelszó-helyreállító linkre kattint. A rendszer automatikusan beilleszti a token és az e-mail címet az űrlapba (az e-mail mezőt letiltva `disabled: true`), így a felhasználónak csak az új jelszót kell megadnia a `resetPasswordForm`-ban.

4.3.5 Adatkezelés és Törzsadatok (CRUD)

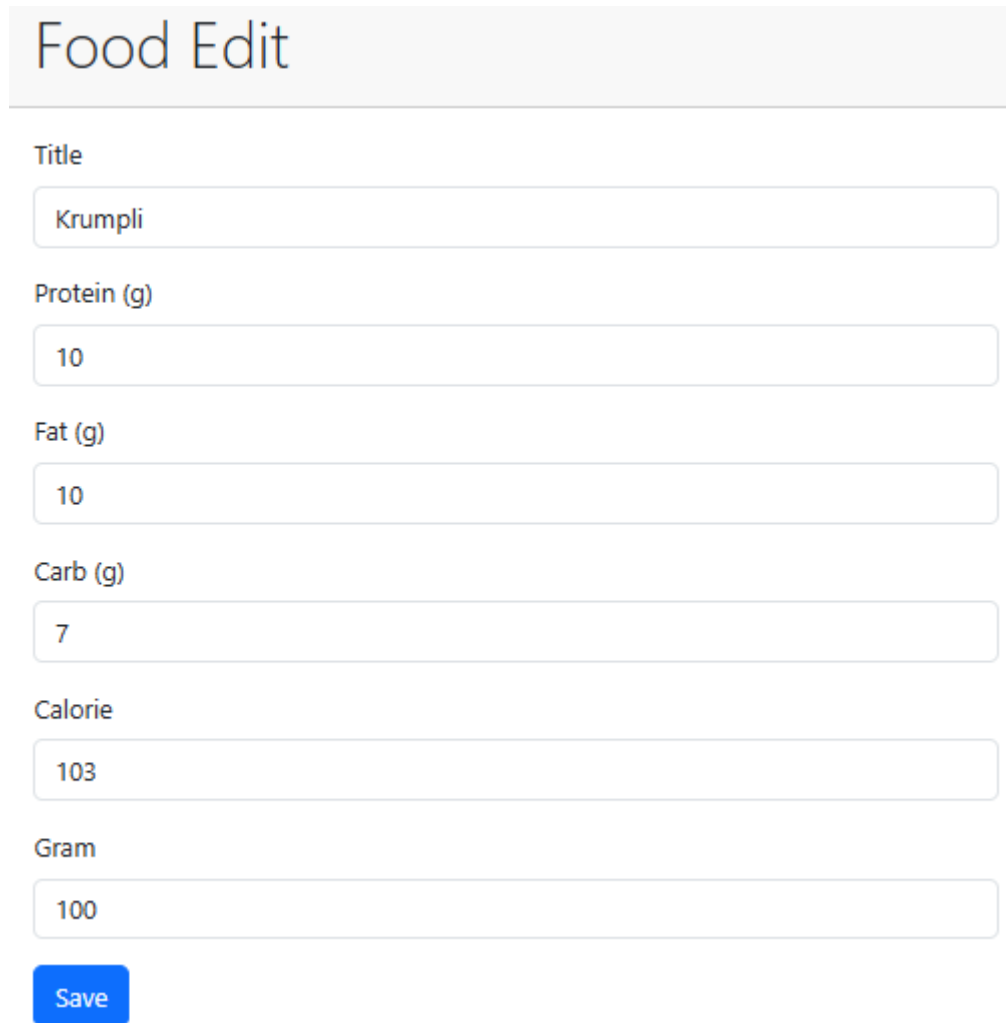
A naplózási funkció működéséhez elengedhetetlenek a törzsadatok. Megvalósítottam a `Food` (Ételek) és `ActivityCatalog` (Tevékenységek) entitások teljes körű kezelését (Létrehozás, Listázás, Szerkesztés, Törlés). Mivel ezek összetett felületek, nem modális ablakokat, hanem dedikált oldalakat használtam a `src/app/pages` könyvtárban.

Listázás és Törlés: A `FoodComponent` és az `ActivityCatalogComponent` felelős az adatok megjelenítéséért. A `loadFoods()` és `loadActivities()` metódusok az inicializáláskor (`ngOnInit`) töltik be az adatokat. A törlési logika (`onDelete`) biztonsági okokból egy megerősítő ablakot (`confirm`) dob fel. Jóváhagyás esetén a komponens meghívja a törlő végpontot, majd siker esetén kliens oldalon, a `filter` metódussal távolítja el az elemet a listából, így nem szükséges újratölteni az egész táblázatot a szerverről.

Létrehozás és Szerkesztés (Kettős célú űrlapok): A kódDuplikáció elkerülése érdekében ugyanazt a komponenst (`CreateFoodComponent`, `CreateActivityCatalogComponent`) használok létrehozásra és szerkesztésre is.

Állapotfelismerés: Az `ngOnInit`-ben vizsgálom az útvonal paramétereit (`route.paramMap`). Ha van `id`, a komponens Szerkesztés módba vált (`editing = true`) és betölti az adatokat.

Mentés: A `createOrUpdate` metódus dönti el az állapot alapján, hogy a `create` (POST) vagy az `update` (PUT) szolgáltatást hívja-e meg.



Food Edit

Title

Krumpli

Protein (g)

10

Fat (g)

10

Carb (g)

7

Calorie

103

Gram

100

Save

7. ábra Food szerkesztési felület

4.3.5.1 Listázás és UI szintű jogosultságkezelés

A `FoodComponent` és az `ActivityCatalogComponent` felelős az adatok táblázatos vagy kártyás megjelenítéséért.

Tekintettel arra, hogy a rendszer specifikációja szerint a vendég felhasználók is megtekinthetik az adatokat, de módosítani csak a regisztrált tagok tudják, egy nézet-szintű védelmet implementáltam a HTML sablonokban. Az Angular `*ngIf` strukturális

direktívájával és az AsyncPipe segítségével közvetlenül feliratkoztam az AccountService.user\$ observable-re.

```
<div *ngIf="(_accountService.user$ | async) as user">
  <button class="btn btn-sm btn-primary"
    (click)="onEdit(food)">Edit</button>
  <button class="btn btn-sm btn-danger"
    (click)="onDelete(food)">Delete</button>
</div>
```

Ez a megoldás biztosítja, hogy a szerkesztő és törlő gombok fizikailag le sem renderelődnek a DOM-ban, ha a felhasználó nincs bejelentkezve, így a felület letisztult marad a vendégek számára is.

Törlési logika (onDelete): A törlés kritikus művelet, ezért a delete metódus meghívása előtt egy natív megerősítő ablakot (confirm) jelenítek meg. Ha a felhasználó jóváhagyja a műveletet, a komponens feliratkozik a FoodClient.deleteFood observable-re. Sikeres válasz esetén nem töltöm újra a teljes listát a szerverről, hanem kliens oldalon, a filter metódussal távolítom el a törölt elemet a foods tömbből, ezzel csökkentve a hálózati forgalmat és javítva a válaszidőt.

4.3.5.2 Létrehozás és Szerkesztés egy komponensben

A karbantartó felületeknél (CreateFoodComponent, CreateActivityCatalogComponent) egy gyakori tervezési mintát alkalmaztam: ugyanaz a komponens szolgálja ki az új elem létrehozását és a meglévő szerkesztését.

Az ngOnInit életciklusban az ActivatedRoute segítségével vizsgálom az URL paramétereket:

Szerkesztés mód: Ha az útvonal tartalmaz id paramétert (route.snapshot.paramMap.get('id')), a komponens editing állapotba kerül. Ekkor azonnal meghívom a GetById szolgáltatást, és a kapott adatokkal (Object.assign segítségével) feltöltöm az űrlap mezőit.

Létrehozás mód: Ha nincs ID, az űrlap üres állapotban inicializálódik.

A mentésért felelős createOrUpdate metódusban egy elágazással döntöm el, hogy a create (POST) vagy az update (PUT) végpontot kell-e meghívni. A kód tisztasága érdekében bevezettem egy beforeSendClean segédfüggvényt is, amely eltávolítja az üres vagy definiálatlan mezőket a DTO-ból a küldés előtt, minimalizálva az adatátvitelt.

4.3.6 Komplex űrlapkezelés: Receptkészítő

Az alkalmazás egyik legösszetettebb űrlapja a CreateRecipesComponent, amely lehetővé teszi, hogy a felhasználó meglévő alapanyagokból állítson össze új ételeket. Itt egy dinamikus, fő-részlet (Master-Detail) kapcsolatot kellett megvalósítanom a memóriában: a recept létrehozása előtt a felhasználó dinamikusan adhat hozzá, módosíthat és törölhet hozzávalókat.

4.3.6.1 Hozzávalók dinamikus kezelése

A recept összetevőit nem közvetlenül az űrlap vezérlőiben, hanem egy lokális tömbben, ingredients néven tárolom (RecipeFoodItemDto[]). A felhasználói interakciót a következő logikai lépésekre bontottam:

Kiválasztás: A komponens betöltésekor (ngOnInit) a loadFoods() metódus feltölti a foodOptions listát. A felhasználó ebből a legördülő menüből választ alapanyagot, és megadja a mennyiséget grammban. A listában való tájékozódást a getFoodTitleById(id) segédfüggvény támogatja, amely az ID alapján megjeleníti az étel nevét.

Hozzáadás/Frissítés (addOrUpdateIngredient): A gombnyomásra a rendszer ellenőrzi, hogy szerkesztésről vagy új hozzáadásról van-e szó.

- Új elem esetén a push() metódussal adom a tömbhöz.
- Meglévő elem szerkesztésekor (editingIngredientIndex nem null) közvetlenül a tömb adott indexű elemét frissítem.

Módosítás és Törlés: A lista minden soránál elhelyeztem egy "Edit" és "Remove" gombot.

- A loadIngredientForEdit(index) visszatölti az adatokat a beviteli mezőkbe, lehetővé téve a korrekciót.
- A removeIngredient(index) a splice() metódussal véglegesen eltávolítja az elemet a listából.

4.3.6.2 Valós idejű számítások (Client-side Calculation)

A felhasználói élmény egyik kulcseleme az azonnali visszajelzés. Nem vártam meg a szerver választ a recept tápértékeinek kiszámításához, hanem az updateNutrition() metódust alkalmaztam.

Ez a függvény minden egyes lista-módosítás (hozzáadás, törlés) után lefut. Végigiterál az ingredients tömbön, és a kiválasztott ételek (foodOptions) ismert tápértékei alapján, a mennyiségekkel súlyozva összesíti a recept teljes kalória-, fehérje-, zsír- és szénhidráttartalmát.

```
updateNutrition(): void {
  this.sumCalorie = 0;
  this.sumProtein = 0;
  this.sumFat = 0;
  this.sumCarb = 0;

  this.ingredients.forEach(ing => {
    const food = this.foodOptions.find(f => f.foodID === ing.foodID);
    if (!food || !ing.quantity) return;

    const factor = ing.quantity / 100;
    this.sumCalorie += (food.calorie || 0) * factor;
    this.sumProtein += (food.protein || 0) * factor;
    this.sumFat += (food.fat || 0) * factor;
    this.sumCarb += (food.carb || 0) * factor;
  });
}
```

Így a felhasználó valós időben látja a receptje "alakulását" a címsorban, még a mentés előtt.

4.3.6.3 Mentés és DTO konverzió

A createOrUpdate metódus végzi a végső összekapcsolást. Létrehozza a RecipeCreateDto objektumot, amely tartalmazza a recept törzsadatait (Cím, Leírás) és a memóriában összeállított ingredients listát. Ezt a komplex objektumot küldöm el a RecipesClient-en keresztül a backendnek, amely tranzakcionálisan menti a receptet és a hozzá tartozó kapcsolótábla-bejegyzéseket.

Edit Recipe 268.00 kcal, 41.00g protein, 13.00g fat, 7.00g carb

Title
Krumppli csirkével

Description
Szíved lelked bele kell rakni.

Add Ingredient
-- Select food -- 0 g Add

Ingredients

Csirkemell	100 g	Edit Remove
Krumppli	100 g	Edit Remove

Save Changes

8. ábra Recipe szerkesztési felület

4.3.6.4 Receptek megjelenítése

A RecipesComponent felelős a kész receptek prezentálásáért. Itt is érvényesül a jogosultságkezelés: bár a kártyákat bárki láthatja, a szerkesztés (/recipes/edit/:id) opció csak a tulajdonosnak vagy bejelentkezett felhasználónak érhető el. A kártyákon (recipes.component.html) már a backend által kiszámolt és tárolt összesített makróértékeket jelenítem meg, így a listázás performancia-igénye minimális.

4.3.7 A Naplózás megvalósítása (Daily Note)

Az alkalmazás funkcionális központja a DailyNoteComponent, amely egy komplex állapotgépet valósít meg: kezeli a dátumváltást, koordinálja a modális ablakok láncolatát, és valós időben frissíti a napi összesítőket. Ebben a fejezetben részletesen bemutatom a komponens belső logikáját és az általa vezérelt folyamatokat.

4.3.7.1 Inicializálás és Dátum-navigáció

A komponens életciklusa az ngOnInit metódussal kezdődik. Itt az elsődleges feladatom az volt, hogy a rendszer képes legyen bármilyen dátum adatait betölteni, ne kizárólag az aznapit. Ezt a célt az Angular ActivatedRoute szolgáltatásával valósítottam meg, feliratkozva a queryParams változásaira.

A navigációt három dedikált metódus támogatja:

A loadDailyNote(date: Date) a központi adatbetöltő függvény. Ez a DailyNoteClient-et használva aszinkron módon lekéri a backendtől a teljes napi

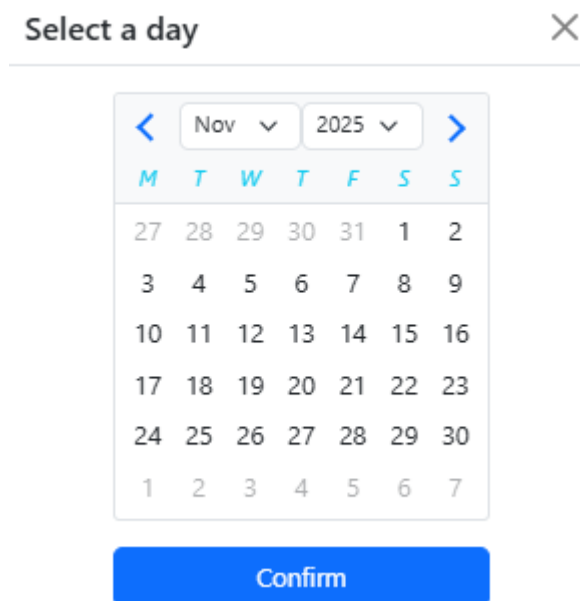
adathalmazt (DailyNoteResponseDto). A válasz megérkezésekor nemcsak a dailyNote változót frissítem, hanem újraszámolom a "burnedCalories" (elégetett kalória) értékét is a calculateBurnedCalories() segédfüggvény meghívásával.

A loadPreviousNote() és loadNextNote() metódusok egyszerű dátum-aritmetikát végeznek: a jelenleg megjelenített dátumhoz képest egy napot vonnak ki vagy adnak hozzá, majd újra meghívják a betöltő logikát.



9. ábra Dátum léptető

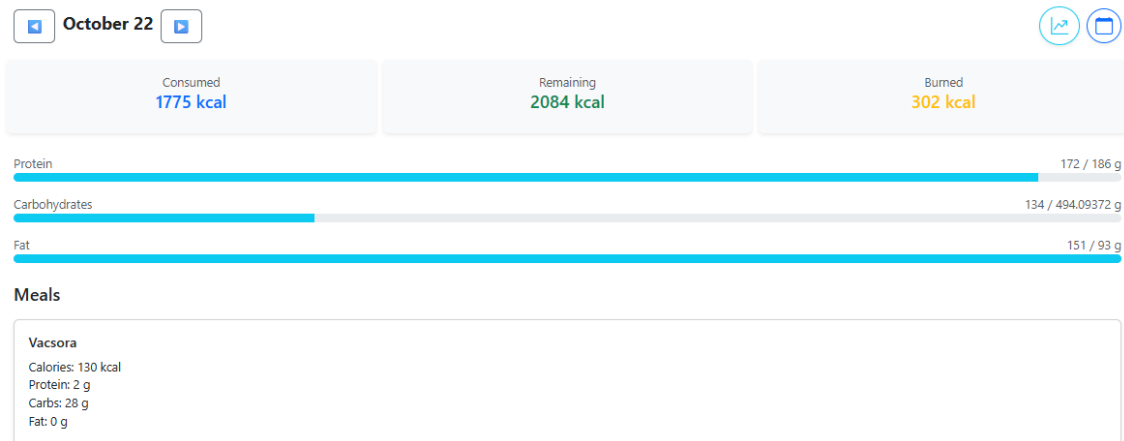
A naptár funkciót az openCalendar() metódus kezeli, amely egy NgbDatepicker komponenst tartalmazó modális ablakot nyit meg. A kiválasztás után a handleCalendarConfirm() végzi el a konverziót az Angular-specifikus dátumformátum és a natív JavaScript Date objektum között, majd frissíti a nézetet.



10. ábra Naptár

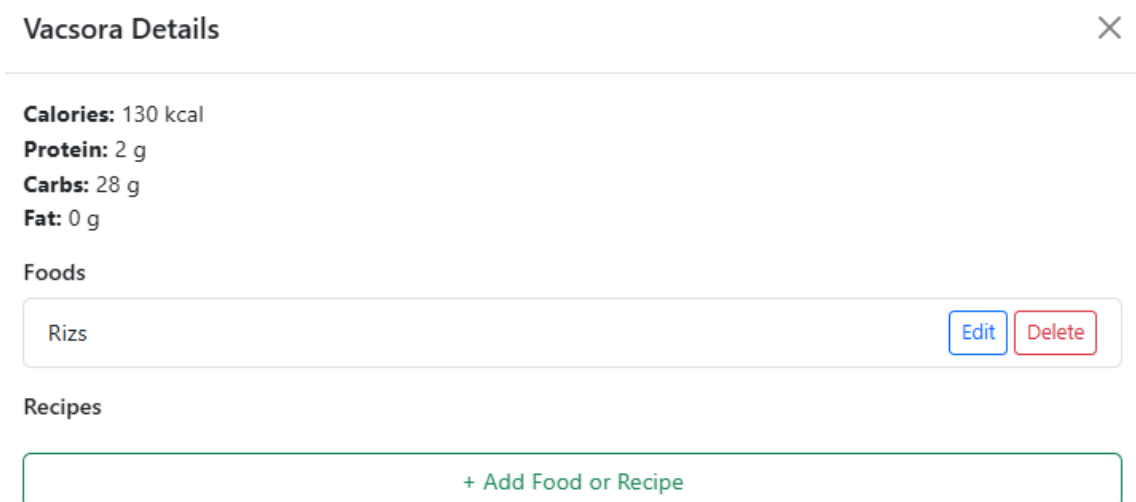
4.3.7.2 Többlépcsős Hozzáadási Folyamat (Modal Chain)

Az új tételek rögzítését a jobb felhasználói élmény érdekében egy háromlépcsős folyamatra bontottam, ahol a DailyNoteComponent vezérli a modális ablakok közötti váltást.



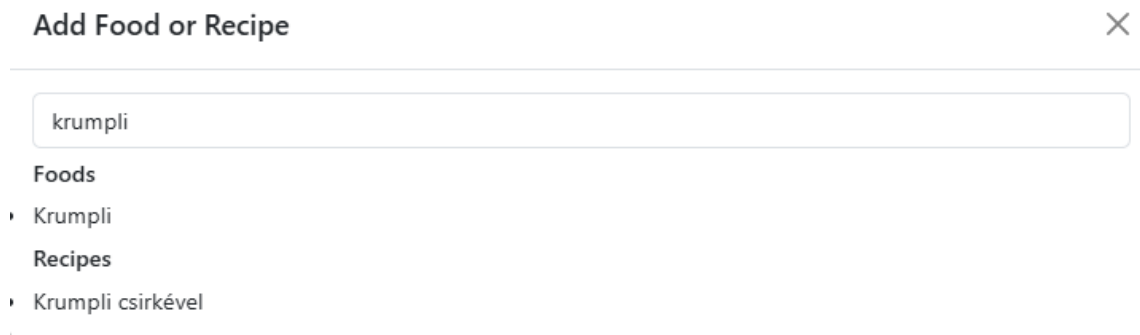
11. ábra DailyNote részlet

1. Keresés és Szűrés (MealItemSearchComponent): A folyamat az `openMealItemSearchModal(mealEntryId)` metódussal indul, amelyet a felhasználó az "Add Food" gombra kattintva vált ki. Ez a függvény elmenti a kiválasztott étkezés (pl. Reggeli) azonosítóját, majd inicializálja a kereső komponenst. A `MealItemSearchComponent`-ben az `onInputChange()` metódus figyeli a gépelést. Azért, hogy elkerüljem a szerver túlterhelését, egy `RxJS Subject`-et és `debounceTime(300)` operátort alkalmaztam, így a keresés csak akkor indul el (`search` metódus), ha a felhasználó 300 milliszekundumig nem gépel. A `clear()` metódus biztosítja, hogy az ablak minden megnyitáskor tiszta találati listával induljon.



12. ábra Vacsora részletes nézet

2. Kiválasztás és Típusfelismerés: A keresőablak egy `itemSelected` eseményt bocsát ki kiválasztáskor. Ezt a fő komponens `handleItemSelected()` metódusa kapja el. Itt egy elágazással vizsgálom meg a kiválasztott elem típusát: amennyiben ételről van szó, bezárom a keresőt és a `FoodQuantityModalComponent`-et nyitom meg; recept esetén pedig a `RecipeQuantityModalComponent`-et.



Add Food or Recipe ✕

Foods

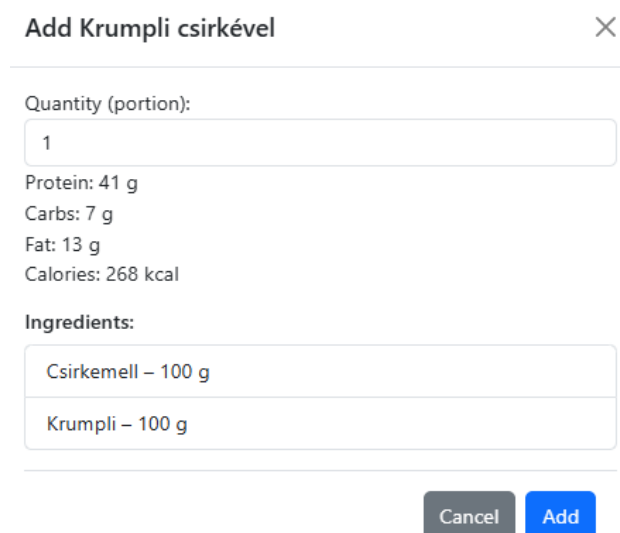
- Krumpli

Recipes

- Krumpli csirkével

13. ábra Kereső mező

3. Mennyiség megadása és Valós idejű számolás: A mennyiség-bekérő modálisok (`FoodQuantityModalComponent`) nemcsak adatbevitelt szolgálnak, hanem számolnak is. Implementáltam az `onQuantityChange()` metódust, amely minden billentyűleütésre (`ngModelChange`) újraszámolja a tápértékeket. Például a fehérje számítása a következők szerint történik: $\text{this.protein} = (\text{this.food.protein} * \text{this.quantity}) / 100$. Ezáltal a felhasználó azonnali visszajelzést kap arról, hogy az általa beírt mennyiség pontosan mennyi tápanyagot tartalmaz.



Add Krumpli csirkével ✕

Quantity (portion):

Protein: 41 g
Carbs: 7 g
Fat: 13 g
Calories: 268 kcal

Ingredients:

Csirkemell – 100 g
Krumpli – 100 g

Cancel Add

14. ábra Mennyiség megadása

4. Mentés: A modális ablak save() metódusa végzi a tényleges API hívást. Létrehozza a megfelelő DTO-t (MealFoodCreateDto), és elküldi a szervernek. Siker esetén a modal egy added kimeneti eseményt (Output) emitál. Erre reagálva a fő komponens onItemAdded() metódusa lezárja az ablakot, eltünteti a szürke hátteret, és a loadDailyNote() meghívásával frissíti a teljes napi nézetet.

Vacsora Details X

Calories: 398 kcal
Protein: 43 g
Carbs: 35 g
Fat: 13 g

Foods

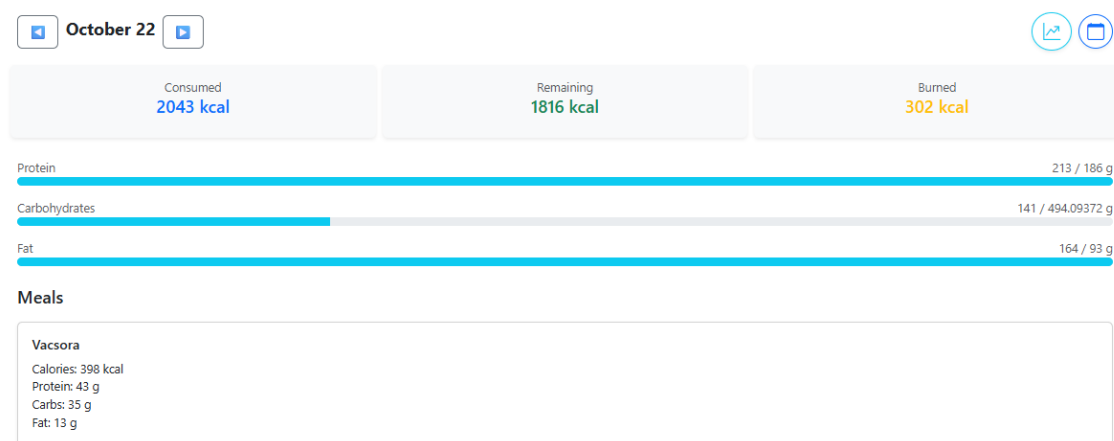
Rizs Edit Delete

Recipes

Krumpli csirkével (1 portion) Edit Delete

+ Add Food or Recipe

15. ábra Vacsora részletes nézet recept hozzáadása után



16. ábra DailyNote részlet nézet recept hozzáadása után

4.3.7.3 Szerkesztés, Törlés és Karbantartás

A naplóbejegyzések utólagos módosítására teljes körű CRUD funkcionalitást biztosítottam, külön metódusokkal kezelve az eltérő entitásokat.

Szerkesztési logika: Amikor a felhasználó a listában egy tételre kattint, az `editMealFood()` vagy `editMealRecipe()` metódus fut le. Ez újra megnyitja a mennyiség-bekérő modált, de egy fontos különbséggel: átadja neki a szerkesztendő tétel azonosítóját (`editingMealFoodId`). A modal `ngOnInit` metódusa ezt érzékeli, és "Létrehozás" helyett "Frissítés" módba kapcsol, az API hívást pedig `create`-ről `update`-re cseréli. A sporttevékenységek módosításáért az `editActivity()` metódus felel. Ez visszatölti a kiválasztott tevékenység adatait a `durationModal`-ba, és engedélyezi a szerkesztést (`isEditMode = true`).

Törlés és Technikai kezelés: A `deleteMealFood()` és `deleteActivity()` metódusok közvetlenül hívják a megfelelő törlő végpontokat, majd a válasz után azonnal frissítik a nézetet. Tekintettel arra, hogy az alkalmazás sok modális ablakot használ, ezért egy `closeAllModals()` technikai segédfüggvényt alkalmaztam. Ennek az a feladata, hogy ablakváltáskor vagy mentéskor manuálisan eltávolítsa a DOM-ból a beragadó modal-backdrop elemeket és a modal-open osztályt, biztosítva, hogy az alkalmazás felülete sose fagyjon le egy beragadt ablak miatt.

4.3.7.4 Testsúly, Aktivitás és Összesítés

A testsúly rögzítését a lehető legegyszerűbbre terveztem. Az `updateWeight()` metódust közvetlenül a beviteli mező `change` (fókuszvesztés) eseményéhez kötöttem, ezáltal amint a felhasználó beírja az új adatot, majd a mentésre kattint a rendszer a háttérben automatikusan elmenti azt.

A sporttevékenységeknél a `saveActivity()` metódus menti el az időtartamot, majd ezt követően lefut a `calculateBurnedCalories()` metódus. Ez a függvény végigiterál a naphoz tartozó összes aktivitáson, és a katalógusadatok alapján kliens oldalon összegzi az elégetett kalóriákat. Ezt az értéket a grafikus felületen, a kalória-folyamatsávon (Progress Bar) egy külön színnel jelenítem meg, vizualizálva a "ledolgozott" energiát.

The screenshot shows a user interface for tracking activities and weight. At the top, there's a header 'Activities' with a '+ Add Activity' button. Below it, there are two activity cards. The first card is titled 'Futás' and shows 'Burned: 272 kcal' and 'Duration: 30 min'. The second card is titled 'Programozás' and shows 'Burned: 55 kcal' and 'Duration: 30 min'. Below these, there's a 'Daily Weight' section with a text input field containing the number '104' and a blue 'Save' button.

17. ábra Aktivitás és Súly felvétel részlet

Végezetül, az adatok hosszútávú elemzéséhez az `openWeightHistoryModal()` metódus példányosítja a `WeightHistoryComponent`-et. Ez a komponens önállóan kéri le a backendtől az idősoros súly adatokat, és egy vonaldiagramon ábrázolja a felhasználó fejlődését.

4.3.8. Profil és Statisztikák

Egy egészségmegőrző alkalmazásnál kulcsfontosságú a személyre szabhatóság és a fejlődés nyomon követése. Ezt a két funkcióterületet a `CreateProfileComponent` és a `WeightHistoryComponent` segítségével valósítottam meg.

4.3.7.5 Profil varázsló és Célok automatikus kezelése

A felhasználói profil nem csupán statikus adatok tárolására szolgál, hanem ez vezérli a napi kalória- és makrotápanyag-számításokat. A `CreateProfileComponent` űrlapját úgy terveztem meg, hogy az első bejelentkezéskor "varázslóként" (wizard) vezesse végig a felhasználót a beállításon, de később bármikor módosítható legyen a profil menüpontból.

Intelligens Cél-detektálás: A felhasználói élmény javítása érdekében nem kérem be explicit módon, hogy a felhasználó fogyni vagy hízni szeretne-e. Ehelyett implementáltam az `updateGoalType()` metódust, amely a megadott Jelenlegi súly (weight) és Célsúly (goalWeight) mezők változásait figyeli (valueChanges).

- Ha a célsúly kisebb, mint a jelenlegi, a rendszer automatikusan Fogyas (Cutting) módba áll.
- Ha nagyobb, akkor Tömegnövelés (Bulking) módba.

- Ha egyenlő, akkor Súlytartás a cél. Ezt az állapotot a felületen egy letiltott (disabled), de informatív mezőben jelenítem meg, így a felhasználó azonnal látja, hogyan értelmezi a rendszer a megadott adatait.

Aktivítási szint: A kalóriaszükséglet (TDEE) pontosításához egy legördülő menüben kínálok fel aktivitási szorzókat (1.2-től 1.9-ig), részletes leírással (pl. "Sedentary", "Very active"), hogy a felhasználó a valósághoz legközelebb álló értéket választhassa.

Adatbetöltés és Mentés: Az ngOnInit során az AccountClient.getProfile() segítségével próbálom betölteni a meglévő adatokat. Ha a szerver 204-es (No Content) választ küld, az azt jelenti, hogy ez egy új regisztráció, így az űrlap üresen inicializálódik. Mentéskor a submit() metódus validálja az űrlapot, és elküldi az UpdateUserProfileDto-t a backendnek.

Edit Your Profile

First Name
Péter

Last Name
Pásztori

Age
25

Height (cm)
186

Weight (kg)
108

Goal Weight (kg)
94

Body Fat (%)
15

Gender
Male

Activity Level
Moderately active (3-5 days/week)

Goal (auto-detected)
Lose weight (Cutting)

Save Profile

18. ábra Profil nézet

4.3.7.6 Adatvizualizáció: Testsúlytörténet grafikon

A fejlődés vizuális visszacsatolása a legmotiválóbb eszköz. A testsúly alakulásának megjelenítésére a Chart.js könyvtárat (és annak idősoros kiegészítőjét, a chartjs-adapter-moment-et) integráltam a WeightHistoryComponent-be.

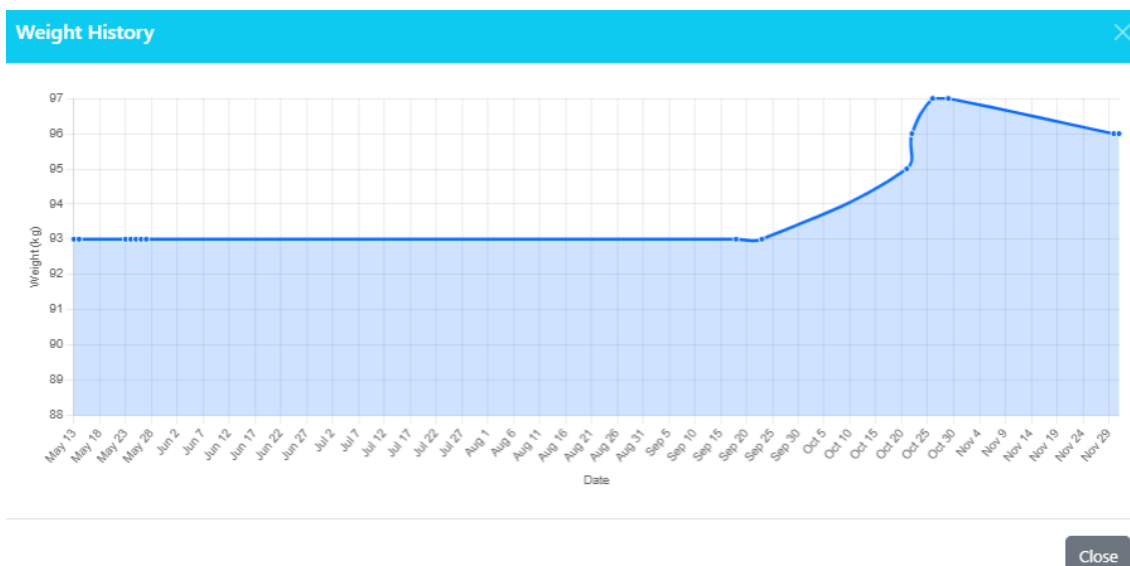
Architektúrális döntés: Azért választottam a Chart.js-t, mert ez egy ipari standard, Canvas-alapú megjelenítő, amely kiváló teljesítményt nyújt nagy adathalmazok esetén is, és rugalmasan testreszabható.

Adatfeldolgozás és Tisztítás: A backendtől kapott nyers DailyNoteResponseDto listát a createChartData() metódusban dolgozom fel.

- Szűrés: Kiszűröm azokat a napokat, ahol a súly 0 vagy érvénytelen, hogy ne torzuljon a grafikon.
- Mapping: Az adatokat a Chart.js által várt { x: timestamp, y: weight } formátumra alakítom át.
- Minimum követelmény: A grafikon csak akkor jelenik meg, ha legalább kettő érvényes mérési pont áll rendelkezésre, ellenkező esetben egy figyelmeztető üzenetet jelenítek meg.

Technikai megvalósítás (Canvas kezelés): Mivel a grafikon gyakran modális ablakban jelenik meg, kritikus volt a renderelés időzítése. Az AfterViewInit élekciklusban, a ChangeDetectorRef.detectChanges() és egy rövid késleltetés (setTimeout) kombinációjával biztosítottam, hogy a <canvas> elem már biztosan létezzen a DOM-ban, mielőtt a Chart.js megpróbálna rajzolni rá. Ezzel elkerültem a modális ablakoknál gyakori "üres grafikon" hibát.

Dinamikus Skálázás: A jobb olvashatóság érdekében a grafikon Y-tengelyét (Súly) nem 0-tól indítom, hanem dinamikusan számolom: a felhasználó legkisebb mért súlyából kivonok 5 kg-ot, és ezt állítom be minimumnak (lineChartOptions.scales['y'].min). Ez biztosítja, hogy a súlyváltozás trendje (a vonal emelkedése vagy süllyedése) látványosan kirajzolódjon, még kis változások esetén is.



19. ábra Profil nézet

4.3.9. Felhasználói élmény (UX) elemek

Egy modern webalkalmazás nemcsak attól jó, hogy működik, hanem attól is, hogy folyamatosan kommunikál a felhasználóval. A fejlesztés során kiemelt figyelmet fordítottam arra, hogy a felhasználó soha ne érezze magát elveszve: minden sikeres vagy sikertelen műveletről visszajelzést adok, és kezelem a navigációs hibákat. Ezeket a funkciókat a SharedModule-ban, újrafelhasználható komponensekként valósítottam meg.

4.3.7.7 Központosított Értesítési Rendszer (Notification)

A rendszerüzenetek (pl. "Sikeres regisztráció", "Hiba történt") megjelenítésére nem mindig a böngésző beépített alert() ablakát, hanem az ngx-bootstrap könyvtár modális szolgáltatását (BsModalService) választottam. A NotificationComponent felelős az üzenetek megjelenítéséért, amely egy esztétikus, felugró ablakot biztosít.

A komponens logikáját úgy terveztem, hogy dinamikusan alkalmazkodjon az üzenet típusához. Egy isSuccess logikai változó vezérli a megjelenést:

- Siker esetén: A fejléc zöld háttérszínt (bg-success) kap.
- Hiba esetén: A fejléc pirosra (bg-danger) vált.

Ezt a dinamikus stílusozást az Angular [ngClass] direktívájával oldottam meg a HTML sablonban. A komponenst az ngx-bootstrap BsModalRef osztályával integráltam,

így az ablak bezárása (`bsModalRef.hide()`) közvetlenül a sablonból vezérelhető, az alkalmazás bármely pontjáról pedig programozottan megnyitható.

4.3.7.8 Újrahasznosítható Validációs Üzenetek

Az űrlapok (Regisztráció, Bejelentkezés, Receptkészítés) validációjakor keletkező hibaüzenetek egységes megjelenítésére létrehoztam a `ValidationMessagesComponent`-et.

Ez a komponens a "dumb component" (buta komponens) tervezési mintát követi: nincs saját üzleti logikája, kizárólag a megjelenítésért felel. Egy `@Input()` dekorátorral ellátott `errorMessages` tömböt vár a szülő komponenstől. A HTML sablonban az `*ngIf` direktívával ellenőrzöm, hogy van-e megjelenítendő hiba, majd az `*ngFor` segítségével listázom ki azokat piros színnel (`text-danger`). Ezzel a megoldással elértem azt, hogy nem kellett minden egyes űrlapnál újra megadnom a hibamegjelenítő HTML struktúrát, jelentősen csökkentve a kódDuplikációt.

4.3.7.9 Hibás útvonalak kezelése (404 Not Found)

A felhasználói élmény része a navigációs hibák elegáns kezelése is. Ha a felhasználó olyan URL-t ír be, amely nem létezik az alkalmazásban, a rendszer nem egy üres oldalt vagy konzolhibát mutat, hanem betölti a `NotFoundComponent`-et.

Ezt a komponens a routing modul `**` (wildcard) útvonalára kötöttem be. A felületen egyértelmű hibaüzenetet és egy "visszaút" gombot helyeztem el, amely a fő oldalra (`/food`) navigálja a felhasználót, így biztosítva, hogy sose kerüljön zsákutcába.

5 Önálló munka értékelése, mérések, tesztelés, eredmények bemutatása

A fejlesztési folyamat záró szakaszában elvégeztem az elkészült rendszer minőségbiztosítási ellenőrzését és teljesítményértékelését. Mérnöki szempontból a legfontosabb feladatom annak igazolása volt, hogy a szoftver nemcsak a funkcionális követelményeket teljesíti, hanem architektúráis szempontból is stabil, karbantartható és hibátűrő. Az értékelést három fő területre csoportosítottam: a kritikus backend folyamatok automatizált tesztelésére, a teljes rendszer integrációs vizsgálatára, valamint a nem-funkcionális követelmények (teljesítmény, skálázhatóság) elemzésére.

5.1 Automatizált tesztelés és kódminőség

A backend üzleti logikájának validálására egy robusztus, automatizált tesztkörnyezetet építettem ki a HealthyAPI.Tests projektben. Mivel az alkalmazás központi eleme a tápanyagok precíz számítása és a különböző entitások (ételek, étkezések, naplók) közötti adatkonzisztencia megőrzése, a „szociális unit tesztek” (sociable unit testing) stratégiáját választottam. Ez a megközelítés lehetővé tette, hogy ne csak izolált metódusokat, hanem a Service réteg osztályainak együttműködését is vizsgáljam valósághű körülmények között.

A tesztkörnyezet technológiai alapjait a .NET ökoszisztéma modern eszközeire helyeztem. A tesztfuttatáshoz az xUnit keretrendszert alkalmaztam, amelynek attribútum-alapú deklarációi átláthatóvá tették a teszteseteket. A kód olvashatóságának növelése érdekében bevezettem a FluentAssertions könyvtárat, amellyel a tesztek elvárásait természetes nyelvi formában (pl. `.Should().Be()`) fogalmazhattam meg, jelentősen megkönnyítve a későbbi karbantartást. A külső függőségek, mint például a fizikai SQL Server, kiváltására az Entity Framework Core InMemory Database szolgáltatóját használtam. Ez a döntés biztosította, hogy minden tesztfuttatás egy tiszta, izolált memóriabeli adatbázison induljon el, kiküszöbölve a környezeti mellékhatásokat és drasztikusan csökkentve a futási időt. A felhasználói kontextus (pl. bejelentkezett User ID) szimulálására a Moq könyvtárat vettem be, amellyel a HttpContextAccessor viselkedését tudtam utánozni.

A tesztesetek fókuszában a „Recalculation Chain” (újra számolási lánc) állt. Részletesen vizsgáltam például a CreateMealFoods_Should_UpdateEntrySums esetet, ahol azt ellenőriztem, hogy egy új alapanyag hozzáadásakor a rendszer helyesen frissít-e az adott étkezés (MealEntry) összesített makróértékeit, és a változás propagálódik-e a napi napló (DailyNote) szintjére. Hasonlóan kritikus volt a DeleteMealFoods forgatókönyv tesztelése, amely garantálja, hogy törlés esetén az értékek pontosan az eredeti állapotra állnak vissza. A receptkezelésnél a mennyiségi szorzók (pl. 1.5 adag étel) helyes skálázását validáltam, biztosítva, hogy a matematikai aggregáció minden esetben hibátlanul fusson le.

5.2 A kész rendszer bemutatása (Felhasználói forgatókönyvek)

A fejlesztés eredményeként egy működőképes, reszponzív webalkalmazás jött létre. Az alábbiakban a rendszer fő funkcióit mutatom be tipikus felhasználói élethelyzeteken (Use Case) keresztül, képernyőképekkel illusztrálva a megvalósult felületet.

5.2.1 Regisztráció és Profilalkotás

A folyamat a „Vendég” szerepkörrel indul. A regisztrációs űrlapon a kötelező adatok megadása után a rendszer azonnal kiküld egy megerősítő e-mailt a Mailjet szolgáltatáson keresztül. (Ide szúrj be egy képet a Regisztrációs űrlapról és/vagy a kapott emailről)

Let's Get Started to HealthyApp

First name
Péter

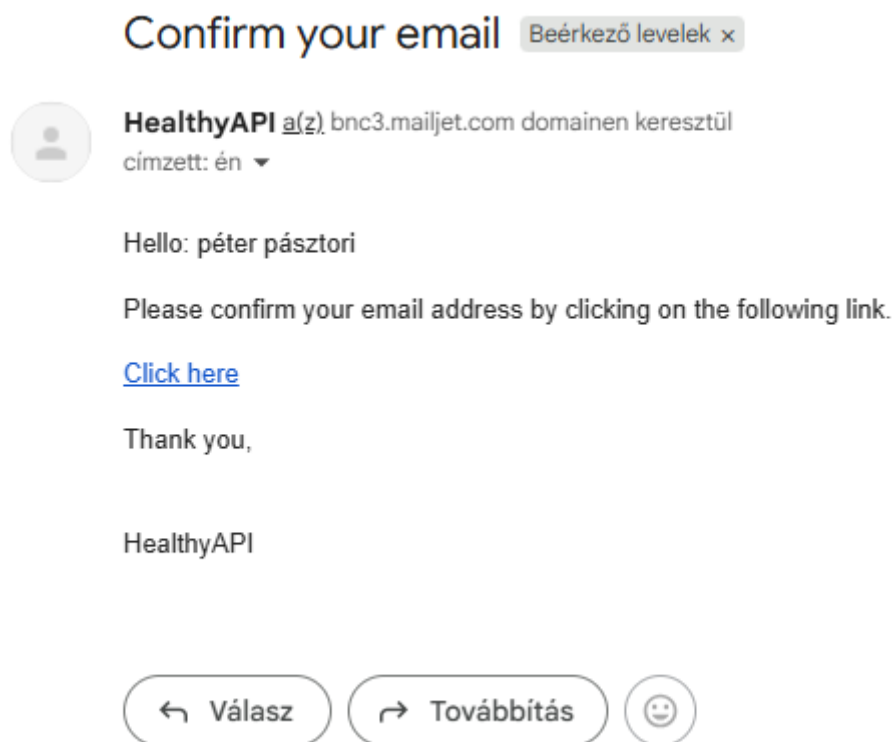
Last name
Pásztori

Email
ppeuroil99@gmail.com

Password
.....

Create Account

20. ábra Regisztrációs űrlap



21. ábra Email üzenet

A sikeres belépés után már szerkesztésre és adatok törlésére is képes a felhasználó. Ha naplóbejegyzést szeretne tenni a felhasználó és nem állította még be a

profilját akkor a „Profil varázsló” fogadja őt. Itt a megadott biometrikus adatok (súly, magasság, kor) és a cél (pl. fogyás) alapján a rendszer a háttérben, a UserProfileService segítségével kiszámolja a BMR és TDEE értékeket. A felületen a felhasználó azonnal látja a számára javasolt napi kalóriakeretet.

5.2.2 Egy teljes nap rögzítése (Naplózás)

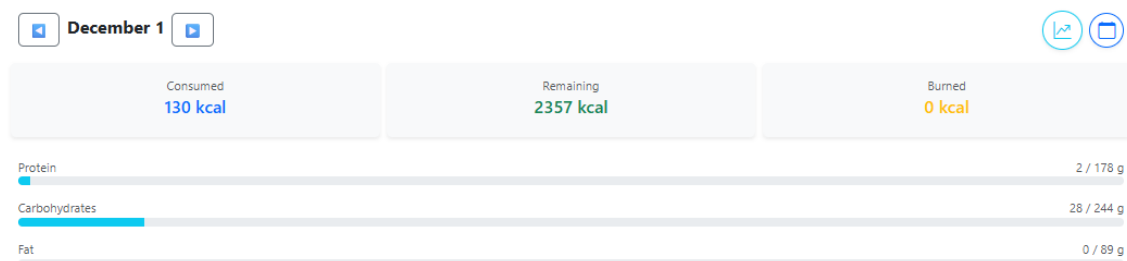
A leggyakoribb használati eset a napi étkezések rögzítése.

Áttekintő nézet: A DailyNote oldalra lépve a felhasználót a mai napra vonatkozó folyamatsávok (Progress Bar) fogadják. Kezdetben ezek üresek.

Étel hozzáadása: Az „Ebéd” kártyára kattintva felugrik a kereső modális ablak. A felhasználó beírja pl. hogy „Rizs”, a rendszer pedig aszinkron módon listázza a találatokat.

Mennyiség megadása: A kiválasztás után a rendszer egy újabb ablakban kéri be a mennyiséget. Itt implementáltam egy valós idejű kalkulátort: ahogy a felhasználó írja be a grammot, a felületen azonnal frissülnek a várható makróértékek.

Eredmény: A mentés gomb megnyomása után az ablak bezárul, és a főoldali folyamatsávok animálva frissülnek az új értékekkel.



21. ábra Progress Bar

5.2.3 Receptkészítés és felhasználás

A rendszer egyik kényelmi funkciója a saját receptek kezelése. A „Receptkészítő” felületen a felhasználó dinamikusan adhat hozzá összetevőket. Bemutatom a „Csirkés rizs” recept létrehozását: a felhasználó felveszi a rizst és a csirkét, a kliensoldali logika pedig valós időben összegzi ezeket. A mentés után ez a recept „saját ételként” megjelenik a keresőben, és egyetlen kattintással hozzáadható bármelyik naphoz.

5.2.4 Hosszú távú követés (Grafikonok)

A rendszer nemcsak a napi, hanem a hosszú távú trendeket is vizualizálja. A „Weight History” modális ablakban a Chart.js könyvtár segítségével egy vonaldiagramon ábrázolom a testsúly alakulását. Ez a funkció igazolja a DailyNote entitásokban tárolt történeti adatok helyes lekérdezését és aggregálását.

5.3 Nem-funkcionális követelmények teljesülése

A mérések és a használat során az alábbi eredményeket tapasztaltam a 3.2-es fejezetben definiált követelményekkel kapcsolatban:

Teljesítmény: Az Angular SPA architektúra és a Scoped élettartamú backend szolgáltatások miatt a navigáció az oldalak között azonnali, a felhasználói élményt nem törik meg újratöltések. Az adatbázis-lekérdezések (pl. havi naptár nézet) válaszüzete a tesztkörnyezetben 200ms alatt maradt.

Reszponzivitás: A Bootstrap Grid rendszer helyes alkalmazását mobil eszközön (Chrome DevTools Device Mode) teszteltem. A menürendszer megfelelően összecsukszik, a táblázatok és kártyák pedig olvashatóan tördelődnek kis kijelzőn is.

Biztonság: A JWT tokenek lejáratási idejének kezelése megfelelően működik; a lejárt token esetén a rendszer automatikusan kijelentkezteti a felhasználót, védve az adatokat.

5.4 Eredmények és a követelmények teljesülése

A fejlesztés eredményeként előállt szoftver sikeresen teljesítette a kitűzött célokat. A funkcionális követelmények tekintetében megvalósult a teljes körű felhasználókezelés (Identity, JWT, Mailjet), a komplex adatmodellel támogatott étel- és receptnyilvántartás, valamint a napi tevékenységek (táplálkozás, sport, testsúly) integrált naplózása.

A nem-funkcionális követelmények terén végzett méréseim alapján a rendszer teljesítménye megfelelő. Az alkalmazott Scoped élettartamú szolgáltatások és az aszinkron (async/await) adatbázis-műveletek biztosítják, hogy a szerver válaszüzete alacsony maradjon, míg a frontend oldalon a „Lazy Loading” (lusta betöltés) technika alkalmazása optimalizálta az indulási sebességet. A biztonsági tesztek igazolták, hogy a token-alapú hitelesítés és a szerveroldali jogosultság-ellenőrzés hatékonyan védi a felhasználói adatokat az illetéktelen hozzáféréstől. A reszponzív felület (Bootstrap Grid)

tesztelése során megbizonyosodtam arról, hogy az alkalmazás mobil és asztali környezetben is egységes felhasználói élményt nyújt.

6 Összefoglaló

Diplomamunkám keretében megterveztem és implementáltam egy modern, többretegű webalkalmazást, amely a felhasználók egészségtudatos életmódját támogatja. A fejlesztés során sikerrel integráltam a szerveroldali ASP.NET Core és a kliensoldali Angular technológiákat, létrehozva egy reszponzív, nagy teljesítményű rendszert.

A rendszer alapjait Code-First megközelítéssel, Microsoft SQL Server adatbázison alakítottam ki, különös figyelmet fordítva a komplex (több-a-többhöz) kapcsolatok kezelésére. A biztonságos működést az ASP.NET Core Identity és a JWT token alapú hitelesítés, valamint a Mailjet e-mail szolgáltatás integrálásával garantáltam. A backend fejlesztésének legfontosabb mérföldköve a „Recalculation Chain” (újraszámolási lánc) architektúra megvalósítása volt, amely biztosítja, hogy a naplóbejegyzések összesített tápértékei minden felhasználói módosítás után (legyen szó receptszerkesztésről vagy étkezés törléséről) valós időben, adatkonzisztencia-hiba nélkül frissüljenek.

A kliensoldali fejlesztés hatékonyságát az NSwag kódgenerátor bevezetésével növeltem, amellyel automatizáltam a frontend és backend közötti típusbiztos kommunikációt. A felhasználói élményt reaktív űrlapokkal és a Chart.js könyvtárral megvalósított dinamikus adatvizualizációval javítottam. A rendszer stabilitását és az üzleti logika helyességét xUnit és InMemory Database segítségével futtatott automatizált integrációs tesztekkel igazoltam.

A jövőben a rendszert egy natív mobilalkalmazással és vonalkód-leolvasó funkcióval tervezem kiegészíteni, amelyek tovább növelnék a mindennapi használhatóságot. A projekt során szerzett legfontosabb szakmai tapasztalatom a „Full Stack” szemlélet elsajátítása volt: megtanultam, hogy a tiszta architektúra, a jól definiált API és az automatizált tesztelés nem csupán elméleti elvárások, hanem a hosszú távon fenntartható és hibátűrő szoftver elengedhetetlen alapfeltételei.

7 Irodalomjegyzék

- [1] „ASP:NET Core áttekintése,” [Online]. Available: <https://learn.microsoft.com/hu-hu/aspnet/core/overview?view=aspnetcore-9.0>.
- [2] „Jwtbearer,” [Online]. Available: <https://www.nuget.org/packages/Microsoft.AspNetCore.Authentication.JwtBearer>.
- [3] „Entity Framework documentation,” [Online]. Available: <https://learn.microsoft.com/hu-hu/ef/>.
- [4] „Email API Overview,” [Online]. Available: <https://dev.mailjet.com/email/guides/>.
- [5] „Nswag,” [Online]. Available: <https://github.com/RicoSuter/NSwag?tab=readme-ov-file>.
- [6] „Angular documentation,” [Online]. Available: <https://angular.dev/overview>.
- [7] „Typescript,” [Online]. Available: <https://www.typescriptlang.org/why-create-typescript/>.
- [8] P. P. Benjámín, „Laboratóriumi információmenedzsment-rendszerek automatikus dokumentáló moduljának fejlesztése”.
- [9] „Visual Studio,” [Online]. Available: <https://visualstudio.microsoft.com/>.
- [10] W3C, „HTML, The Web’s Core Language,” [Online]. Available: <http://www.w3.org/html/>. [Hozzáférés dátuma: 20 október 2015].
- [11] „OpenAPI,” [Online]. Available: <https://swagger.io/resources/open-api/>.